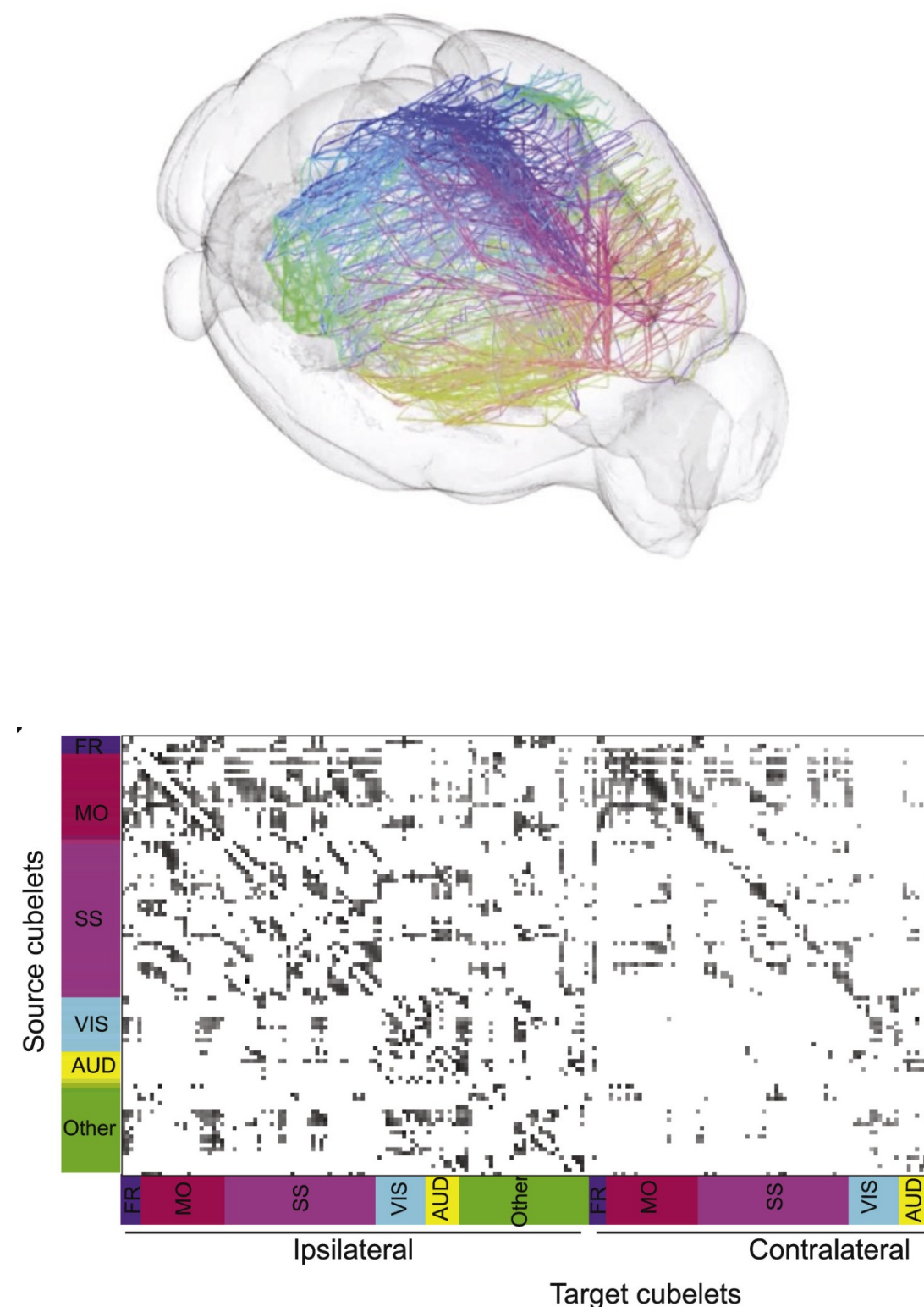


The structure-function mapping in low-rank recurrent neural networks

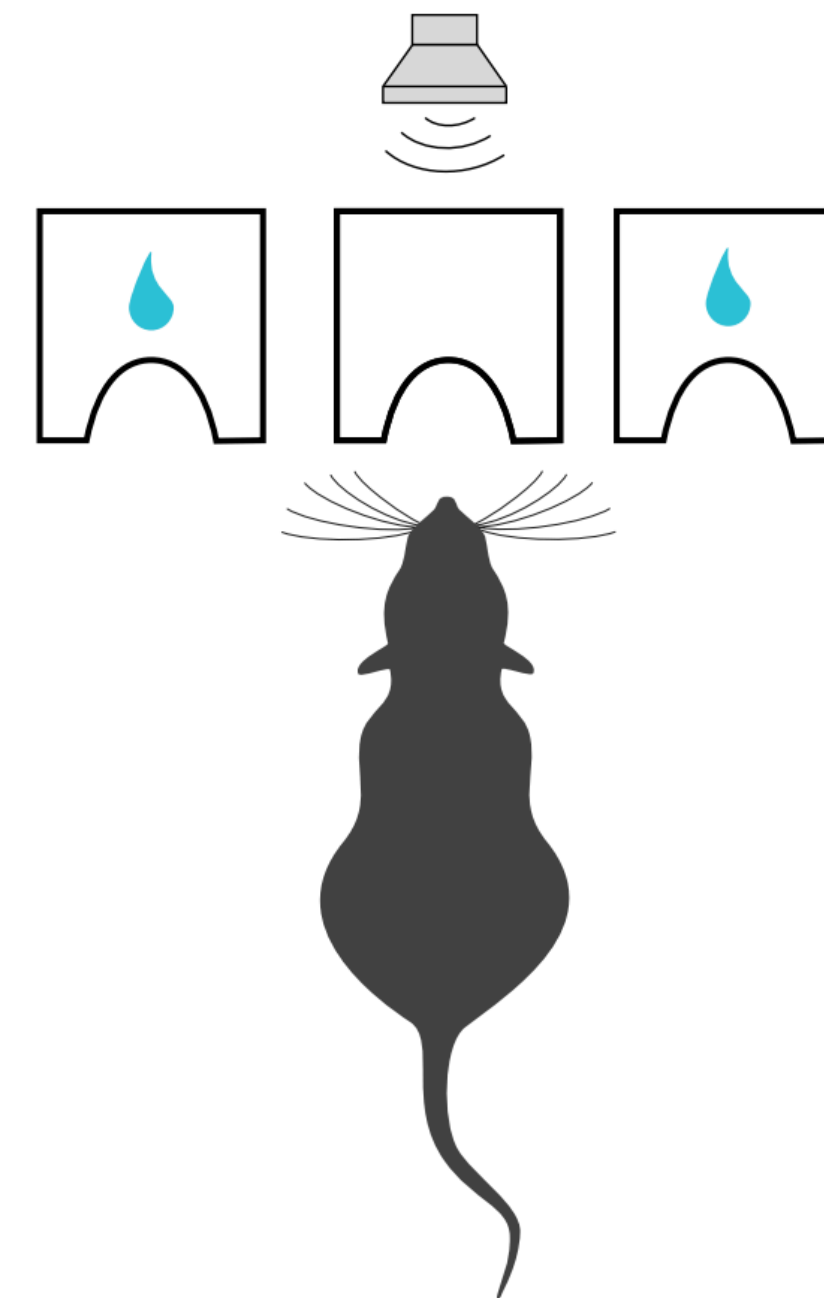
Srdjan Ostojic
Ecole Normale Supérieure Paris

Structure



[Huang et al 2020]

Function



[Raposo et al 2014]

Recurrent Neural Networks

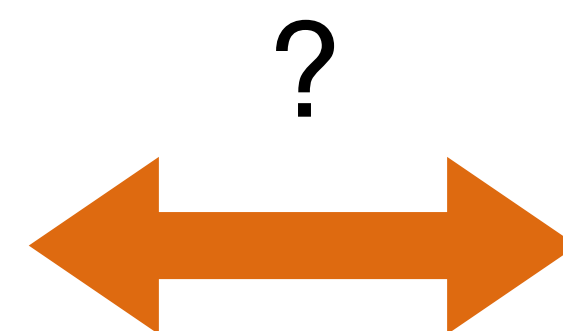
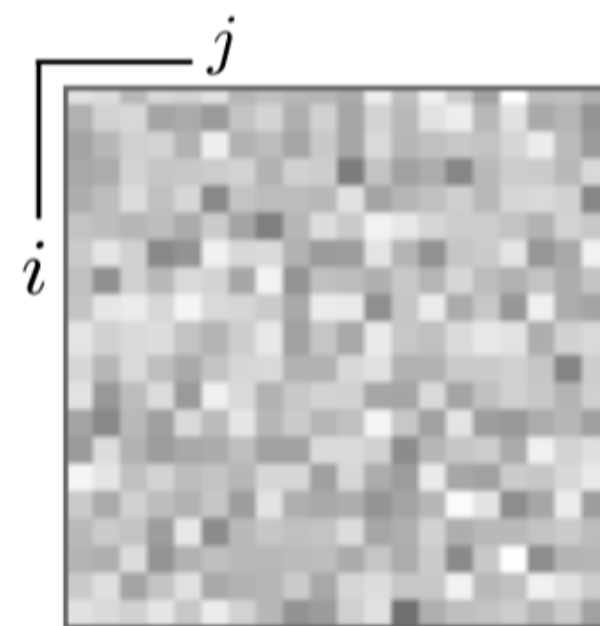
$$\tau \frac{dx_i}{dt} = -x_i + \underbrace{\sum_{j=1}^N J_{ij} \phi(x_j)}_{\text{recurrent input}} + \underbrace{u(t)I_i}_{\text{external input}}$$

coupling strength from unit j to i

rate of unit j
 $r_j(t) = \phi(x_j)$

→ Connectivity matrix

Structure



Function

Input

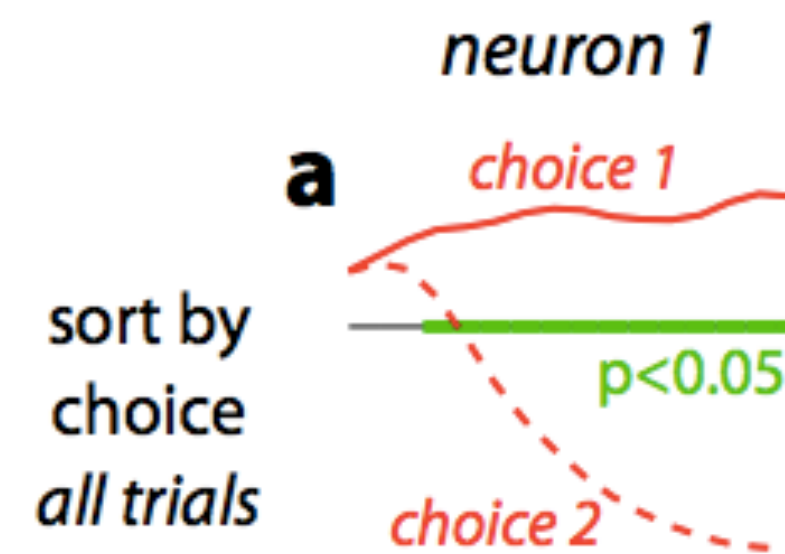


Output

Neural computations



[Mante, Sussillo et al Nature 2013]



activity

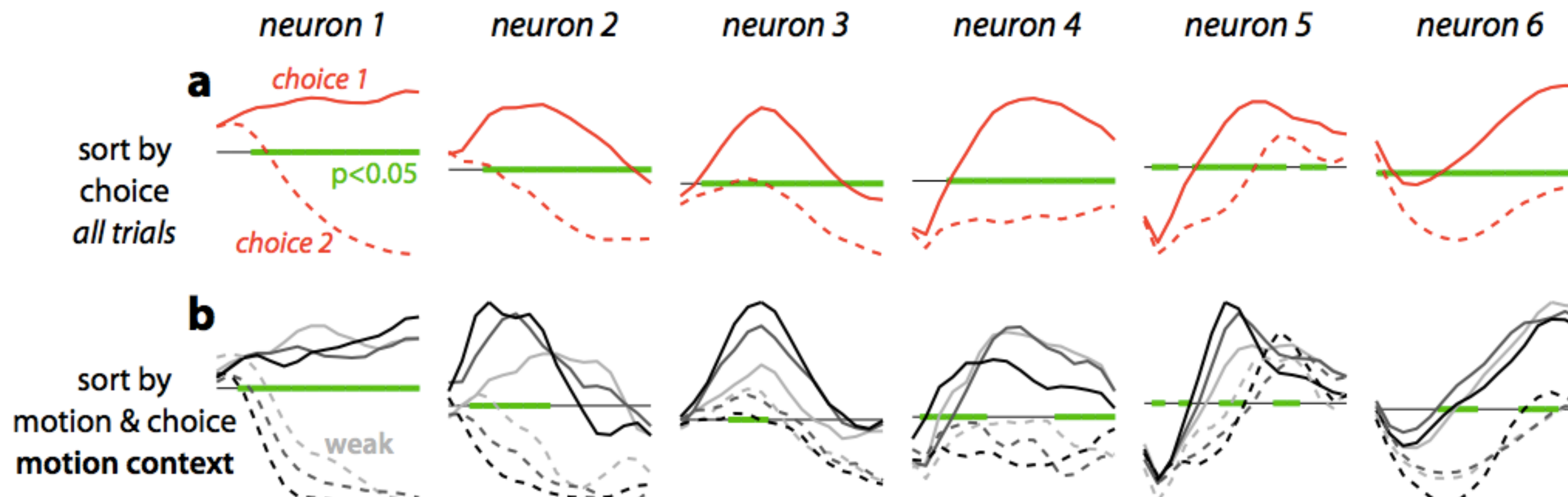
time

Neural computations



[Mante, Sussillo et al Nature 2013]

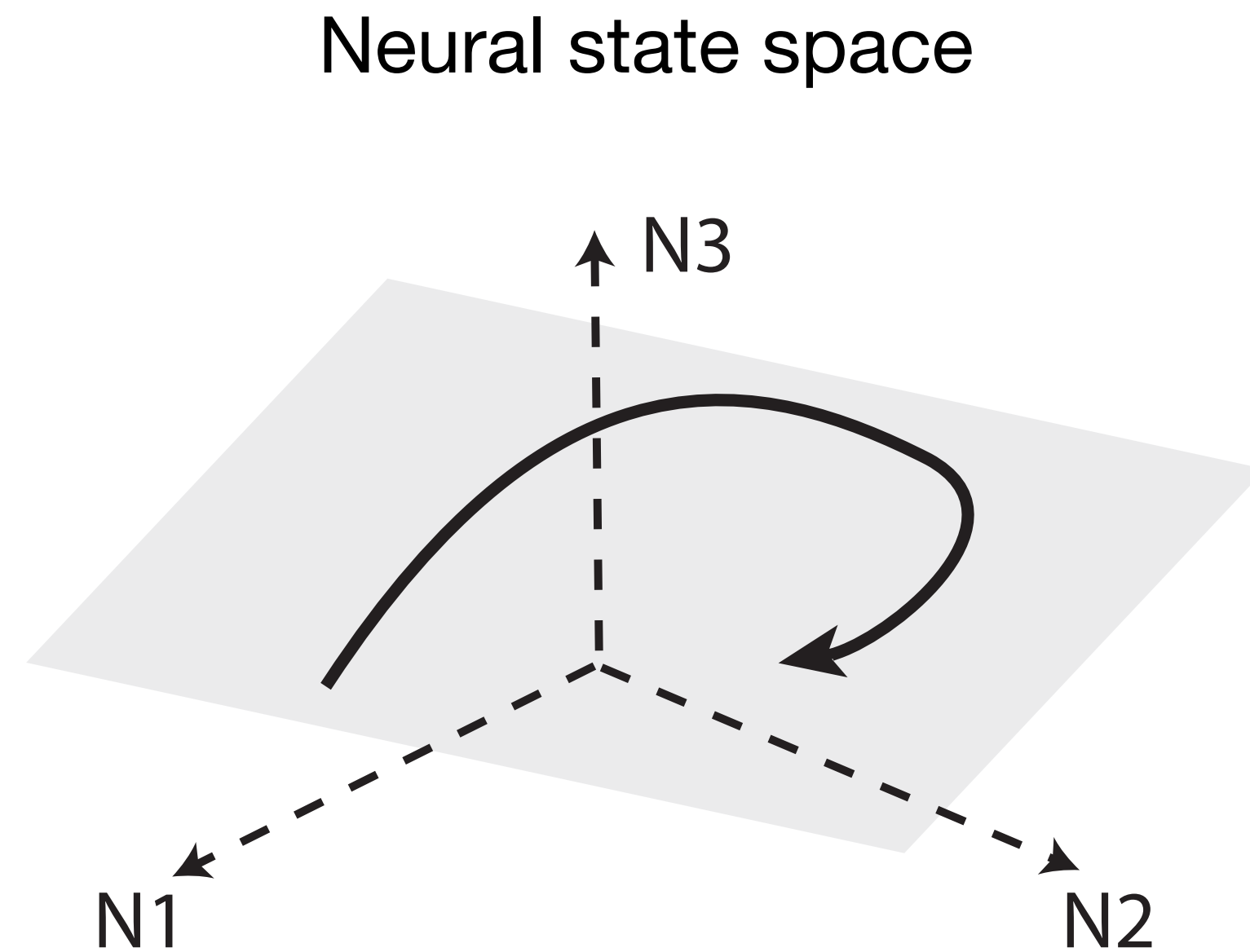
activity
time



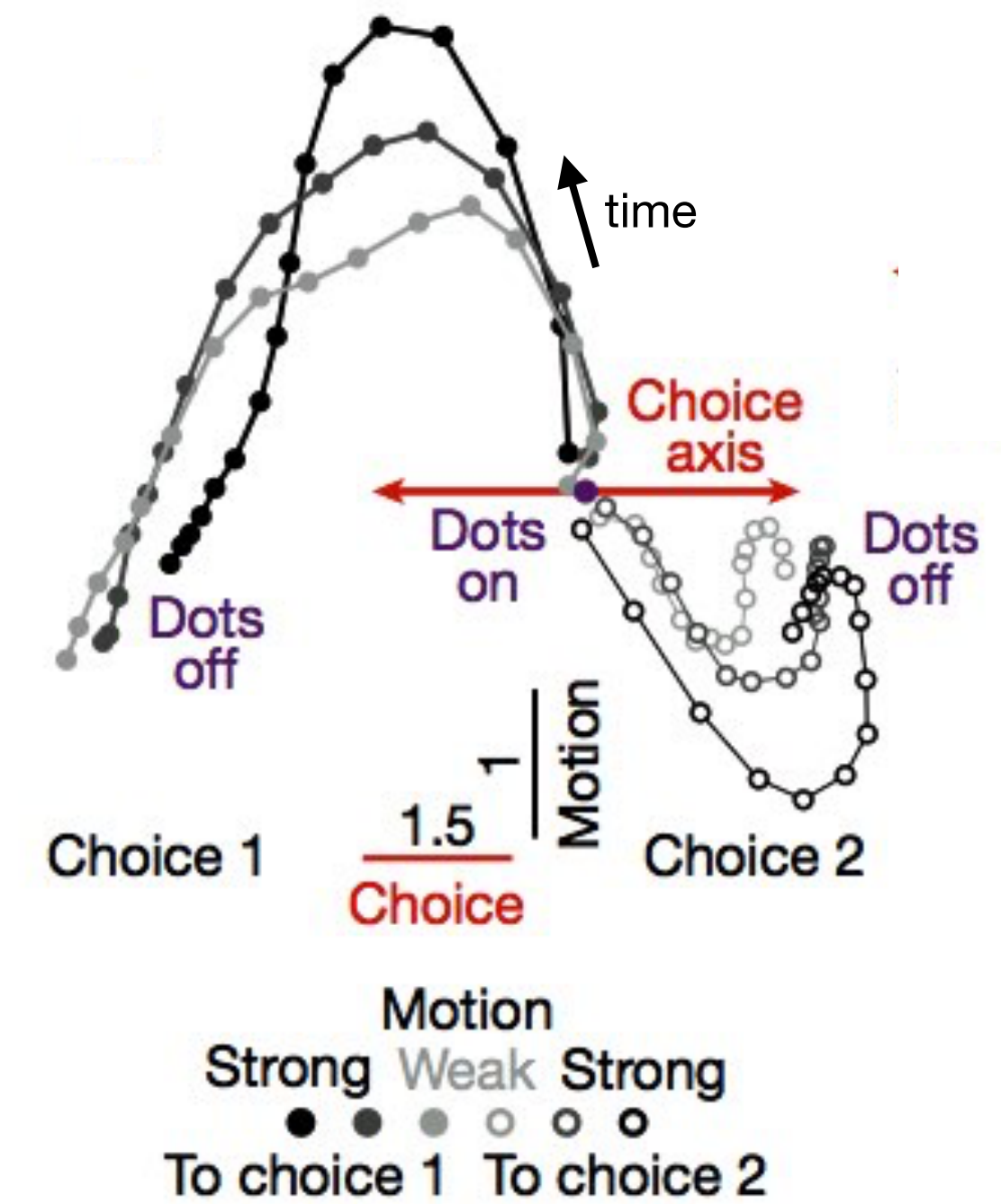
➔ **Mixed selectivity**

[Rigotti et al Nature 2013]

Collective dynamics



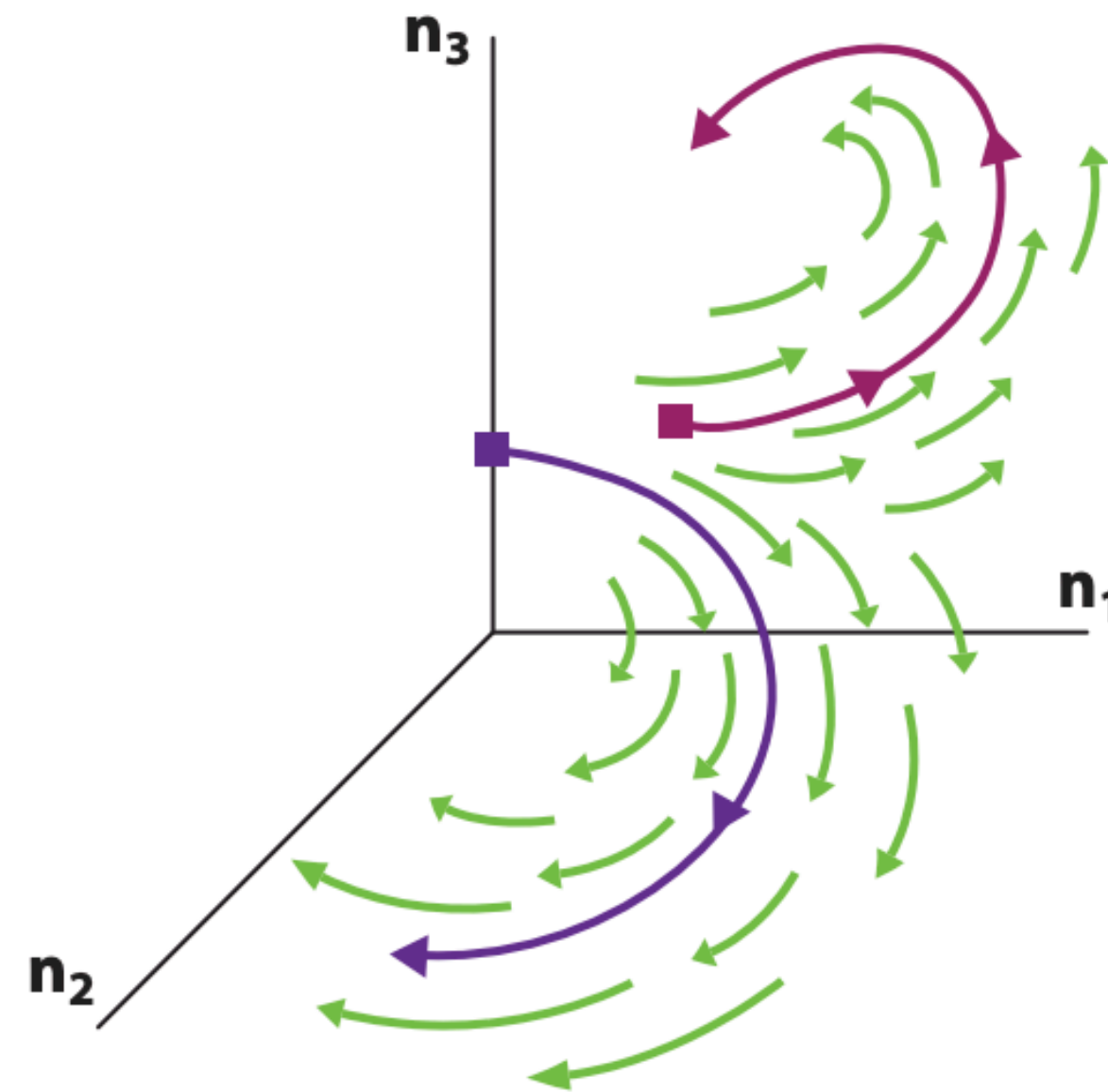
two-dimensional
projection



[Machens et al 2010, Cunningham & Yu 2014, Gallego et al 2017 ...]

[Mante, Sussillo et al 2013]

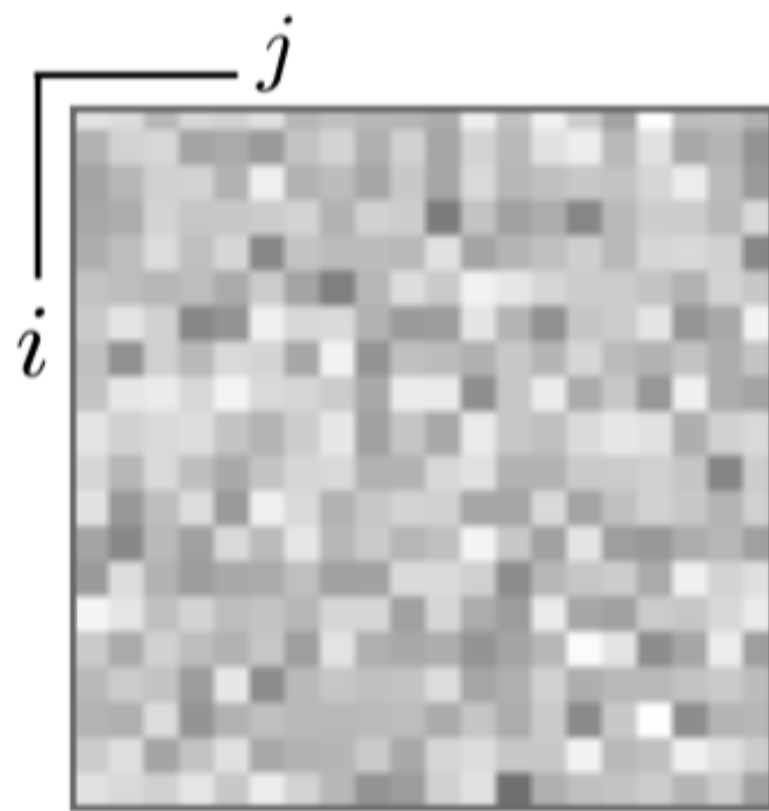
Computations through Population dynamics



$$\frac{d\kappa}{dt} = F(\kappa)$$

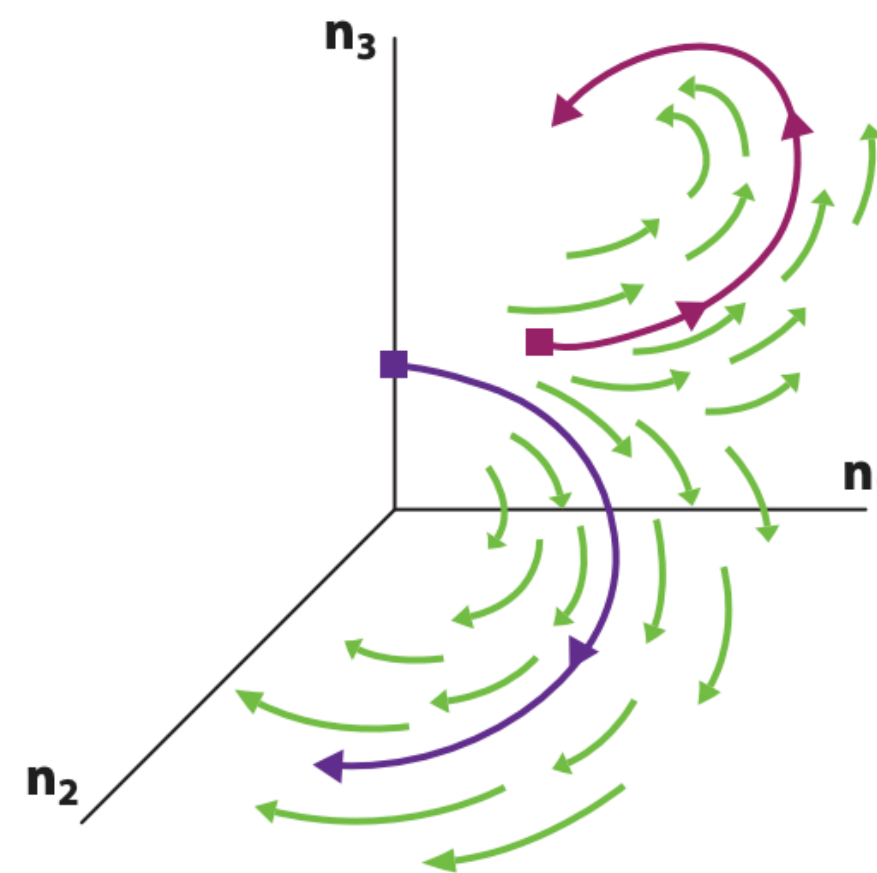
➔ **Dynamical Systems of Latent Variables**

Structure



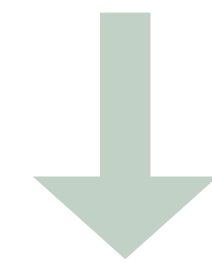
Latent dynamics

$$\frac{d\kappa}{dt} = F(\kappa)$$

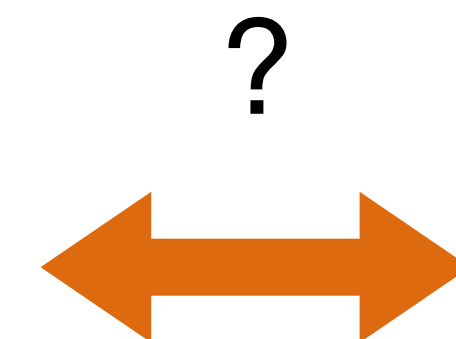
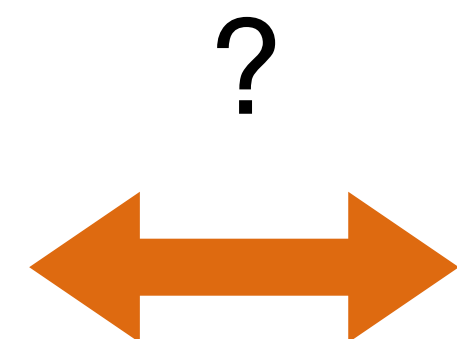


Function

Input



Output



What is the relevant structure in connectivity?

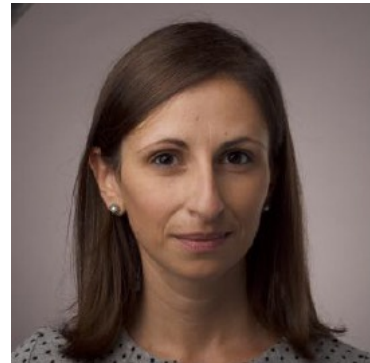


Need simplified models!

Outline

1. Theoretical framework: low-rank recurrent networks
2. Reverse-engineering trained networks

Acknowledgements



Francesca Mastrogiuseppe (Champalimaud) [Mastrogiuseppe & SO Neuron 2018]



Manuel Beiran (Columbia) [Beiran et al, Neural Comp 2021]



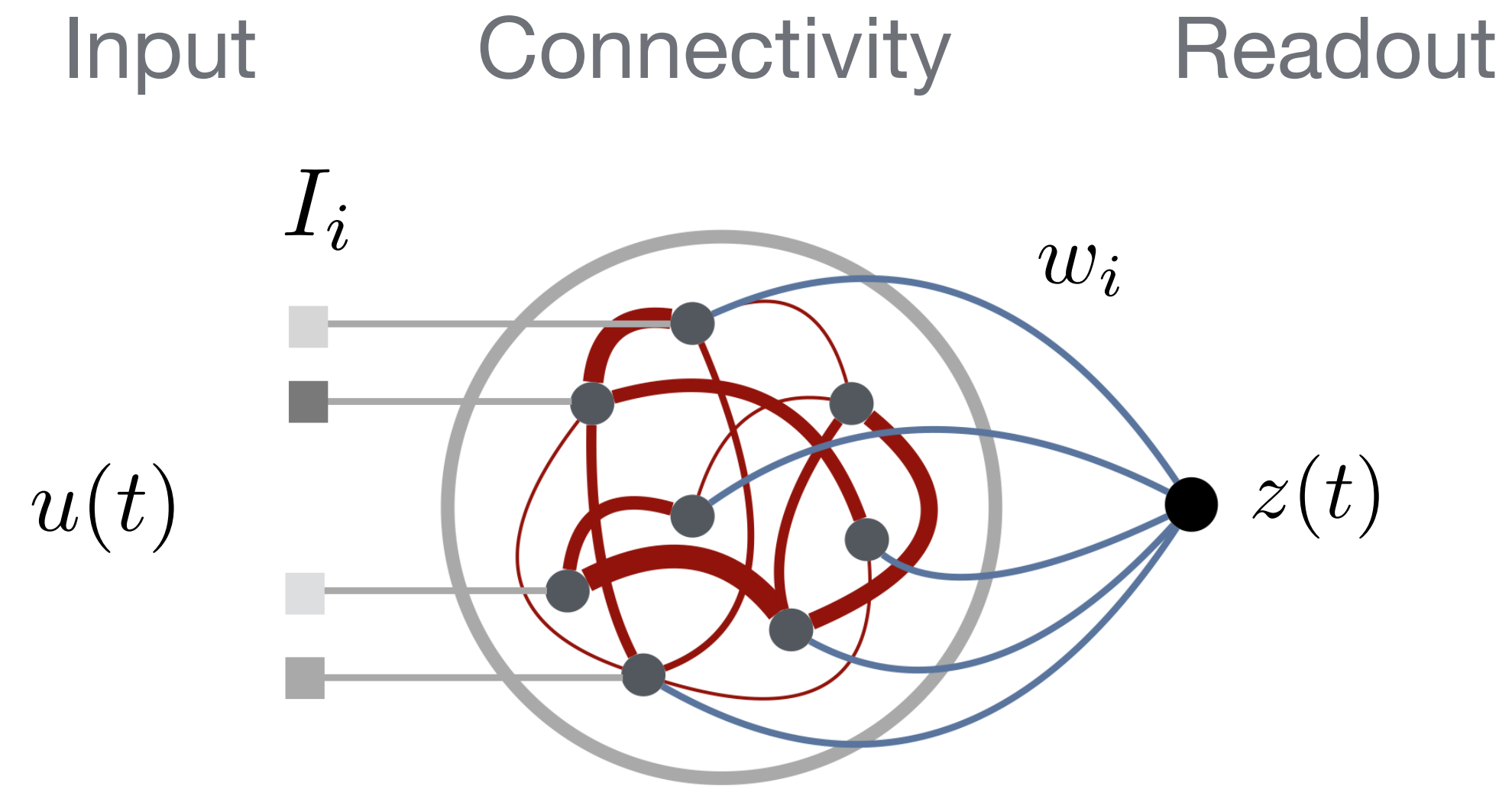
Alexis Dubreuil (Bordeaux) [Dubreuil*, Valente* et al, Nat Neuro 2022]



Adrian Valente (ENS) [Valente, Pillow &SO, NeurIPS 2022]



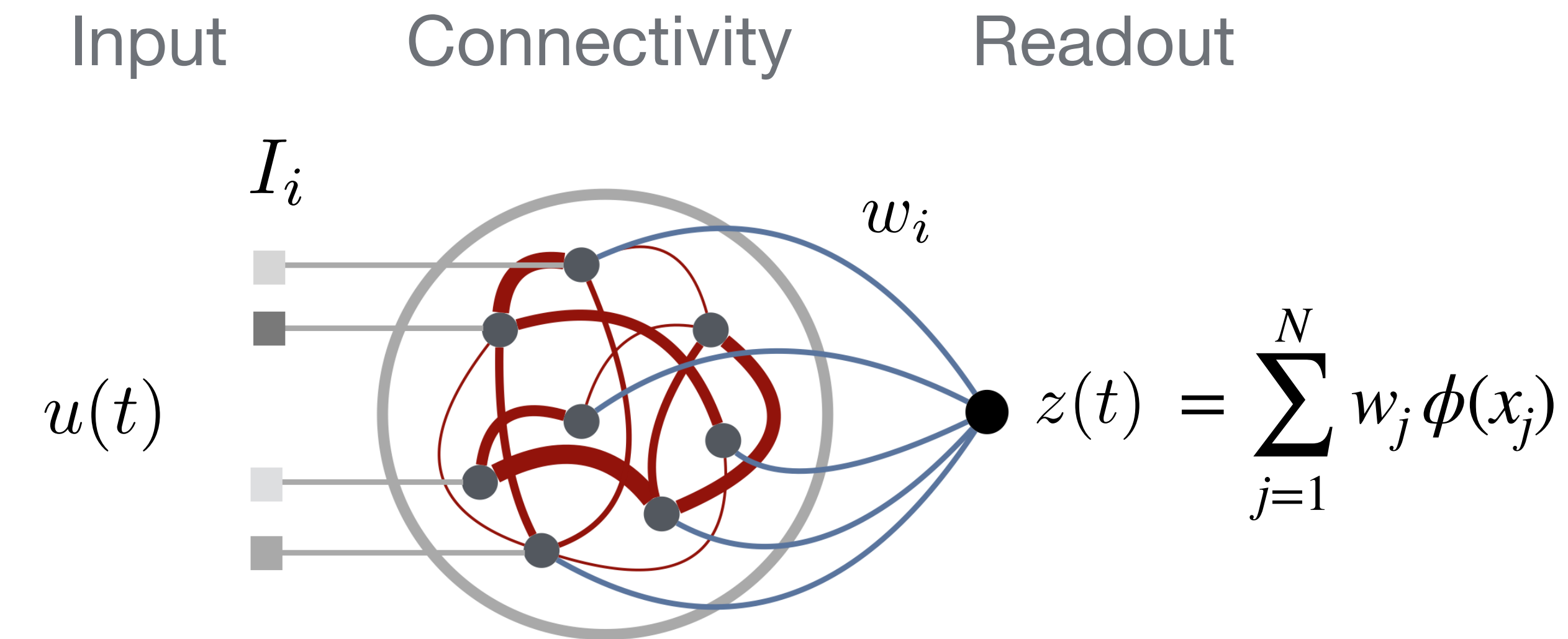
Recurrent Neural Networks



$$\tau \frac{dx_i}{dt} = -x_i + \sum_{j=1}^N J_{ij} \phi(x_j) + u(t)I_i$$

[Sompolinsky, Crisanti, Sommers 1988]

Recurrent Neural Networks



$$\tau \frac{dx_i}{dt} = -x_i + \underbrace{\sum_{j=1}^N J_{ij} \phi(x_j)}_{\text{recurrent input}} + \underbrace{u(t)I_i}_{\text{external input}}$$

[Sompolinsky, Crisanti, Sommers 1988]

coupling strength from unit j to i

rate of unit j

→ Connectivity matrix

$$r_j(t) = \phi(x_j)$$

$$\phi(x) = \tanh(x)$$

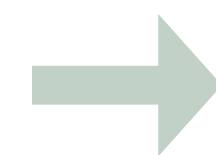
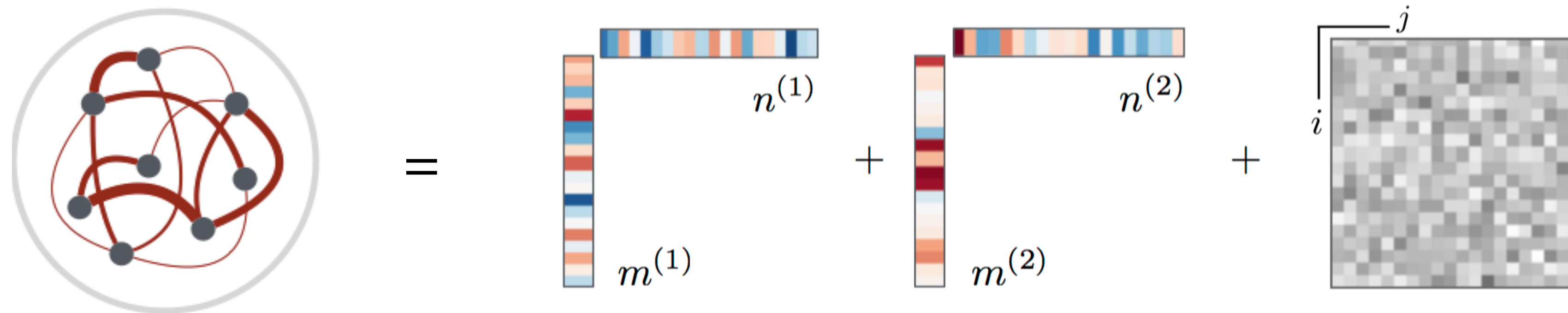
Low-rank RNNs

Connectivity

$$J_{ij} = m_i^{(1)} n_j^{(1)} + m_i^{(2)} n_j^{(2)} + \dots + m_i^{(r)} n_j^{(r)} + g\chi_{ij}$$

low-rank structure

random



connectivity vectors

Low-rank RNNs

$$J_{ij} = m_i^{(1)} n_j^{(1)} + m_i^{(2)} n_j^{(2)} + \dots + m_i^{(r)} n_j^{(r)} + g\chi_{ij}$$

Why?

- classical models are based on low-rank connectivity:
 - Attractor networks [Hopfield 1982]
 - Ring model [Ben Yishai et al 1995], line integrator [Seung et al 2000]
 - Balanced EI networks [van Vreeswijk & Sompolinsky 1996, 1998, Amit & Brunel 1998]
 - Echo-state and Force [Jaeger and Haas 2004, Sussillo and Abbott 2009]
 - Neural engineering [Eliasmith 2005]
 - Spike-based predictive coding [Boerlin, Machens and Deneve 2013]
- directly related to statistical models for dimensionality reduction [Valente et al 2022]
- learning/training leads to low-rank structure [Schuessler et al, NeurIPS 2020]
- **mathematically tractable/interpretable + highly expressive**

[Mastrogiuseppe & SO 2018, Schuessler et al, 2020a]

Low-rank RNNs

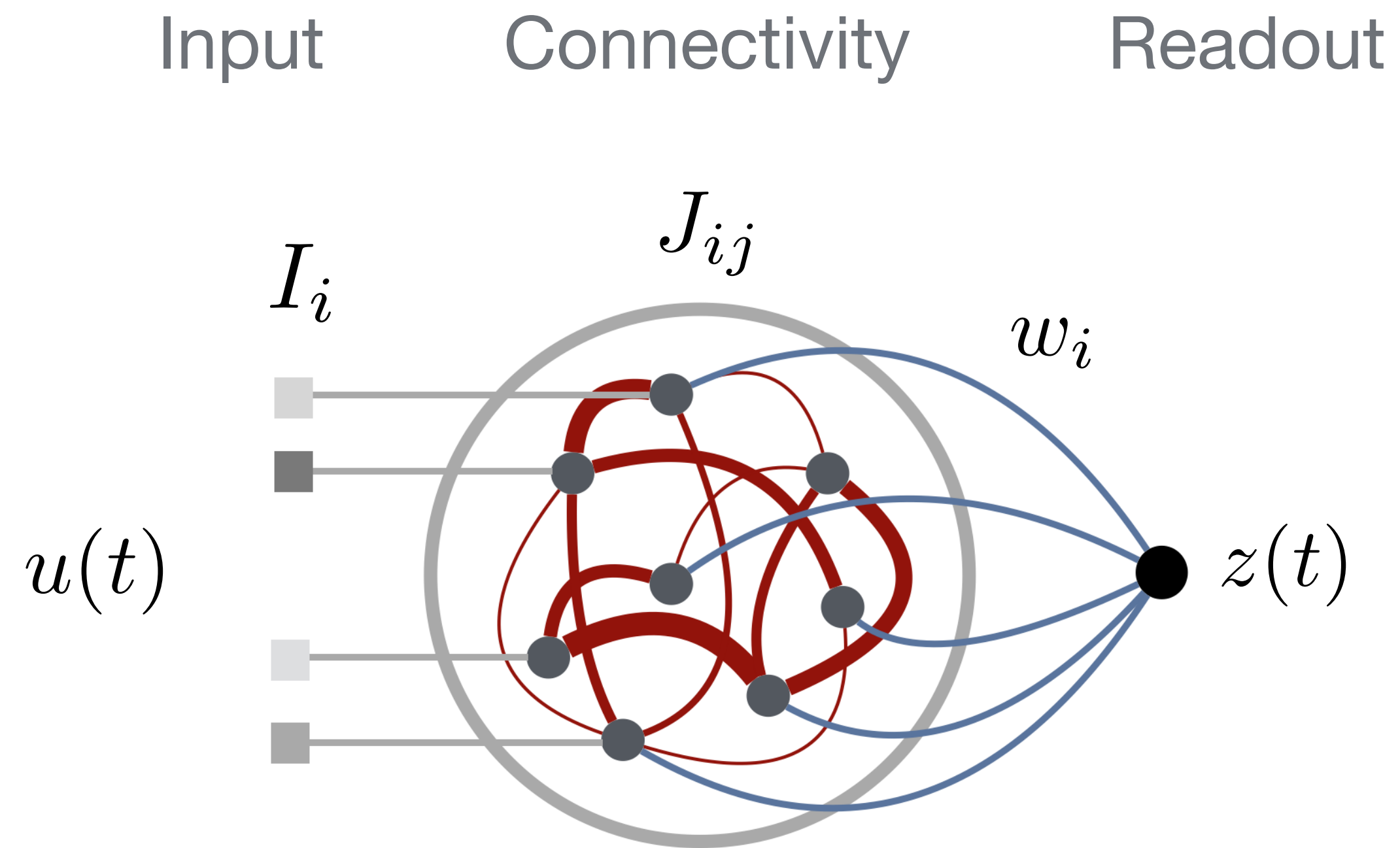
$$J_{ij} = m_i^{(1)} n_j^{(1)} + m_i^{(2)} n_j^{(2)} + \dots + m_i^{(r)} n_j^{(r)} + \cancel{g\chi_{ij}}$$

Why?

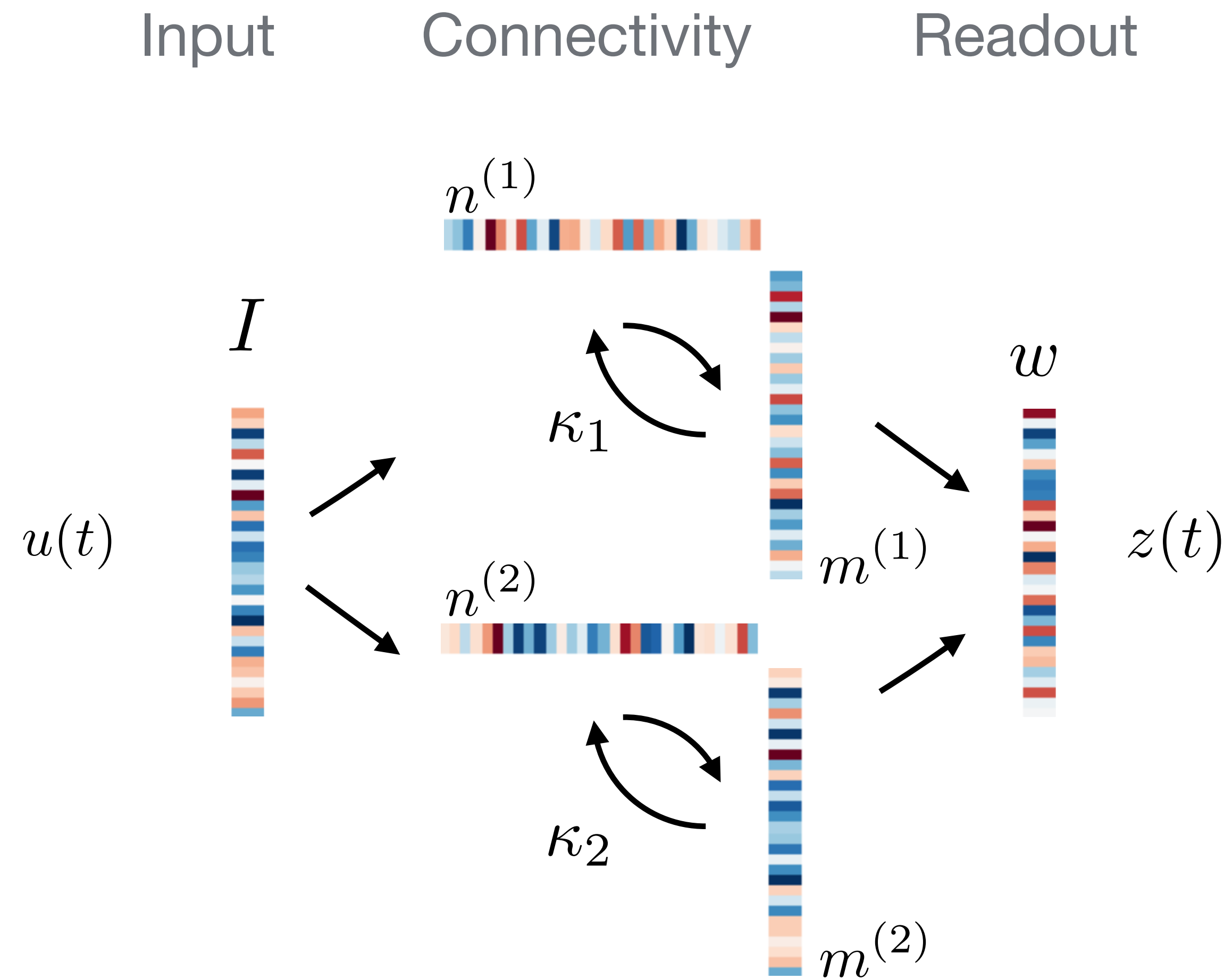
- classical models are based on low-rank connectivity:
 - Attractor networks [Hopfield 1982]
 - Ring model [Ben Yishai et al 1995], line integrator [Seung et al 2000]
 - Balanced EI networks [van Vreeswijk & Sompolinsky 1996, 1998, Amit & Brunel 1998]
 - Echo-state and Force [Jaeger and Haas 2004, Sussillo and Abbott 2009]
 - Neural engineering [Eliasmith 2005]
 - Spike-based predictive coding [Boerlin, Machens and Deneve 2013]
- directly related to statistical models for dimensionality reduction [Valente et al 2022]
- learning/training leads to low-rank structure [Schuessler et al, NeurIPS 2020]
- **mathematically tractable/interpretable + highly expressive**

[Mastrogiuseppe & SO 2018, Schuessler et al, 2020a]

Low-rank RNNs



Low-rank RNNs



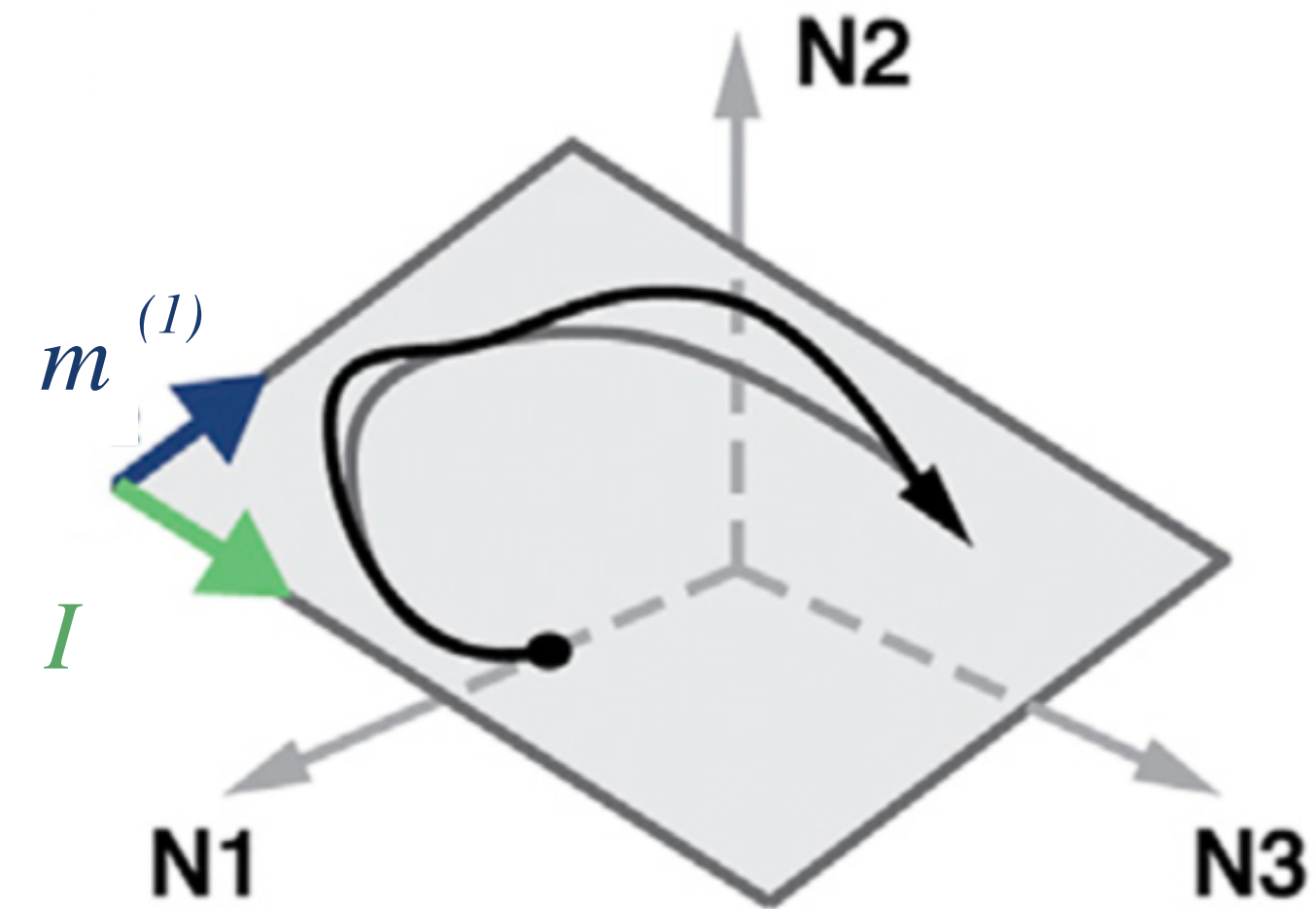
Low-rank theory

Population activity

$$x_i = \underbrace{\kappa_1 m_i^{(1)} + \kappa_2 m_i^{(2)} + \dots + \kappa_r m_i^{(r)}}_{\text{recurrent}} + \underbrace{u(t) I_i}_{\text{input-driven}}$$

→ $\kappa_1 \dots \kappa_r$: **latent variables**

$\mathbf{m}^{(1)} \dots \mathbf{m}^{(r)}$: principal directions



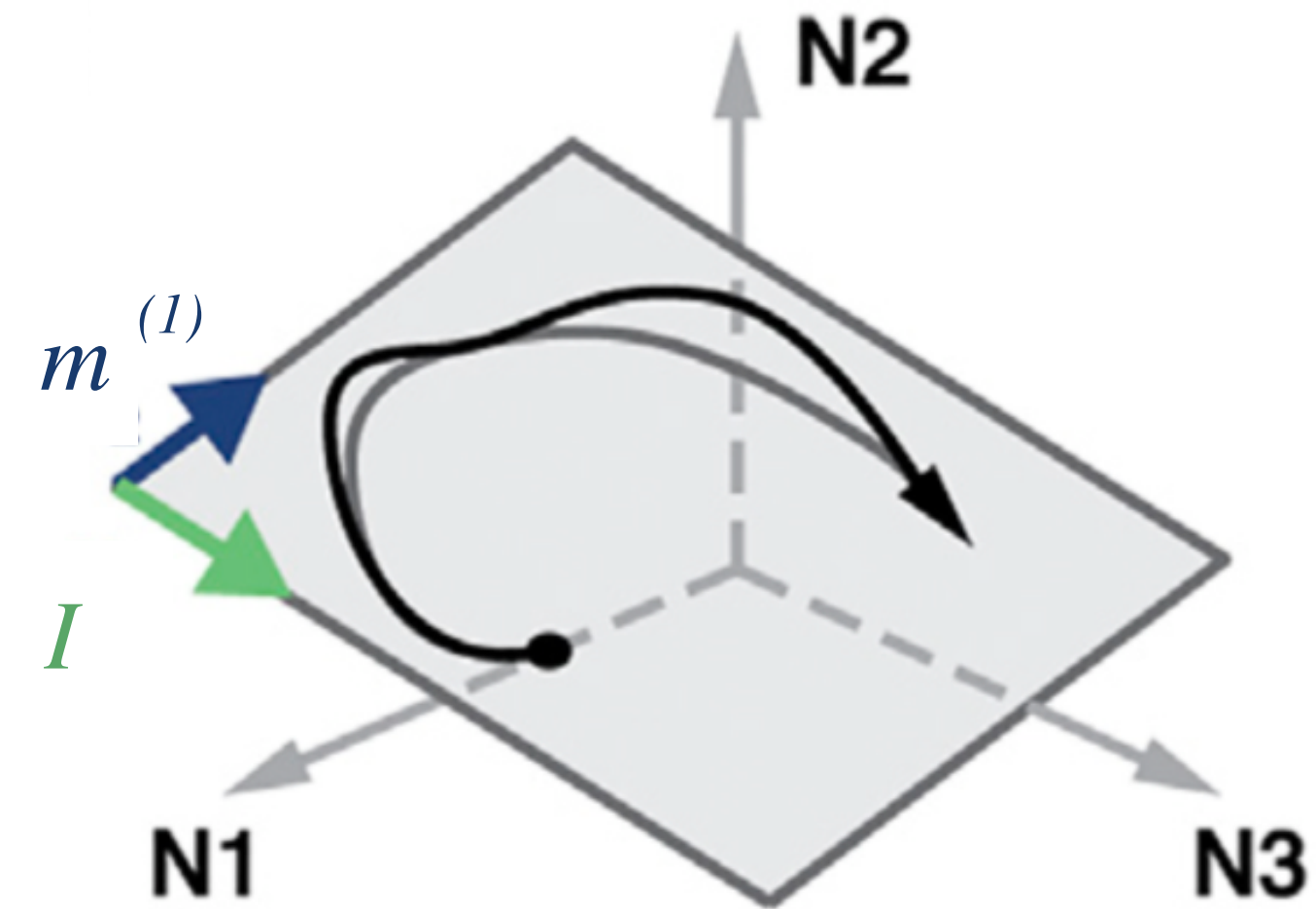
Low-rank theory

Population activity

$$x_i = \underbrace{\kappa_1 m_i^{(1)} + \kappa_2 m_i^{(2)} + \dots + \kappa_r m_i^{(r)}}_{\text{recurrent}} + \underbrace{u(t) I_i}_{\text{input-driven}}$$

→ $\kappa_1 \dots \kappa_r$: **latent variables**

$\mathbf{m}^{(1)} \dots \mathbf{m}^{(r)}$: principal directions



Latent dynamics

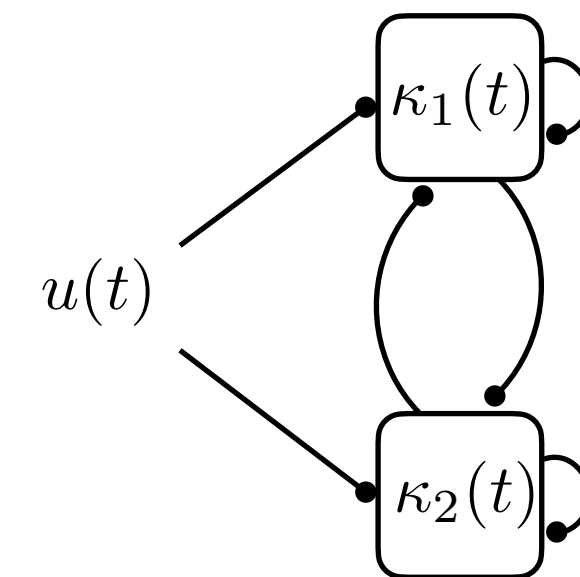
$$\frac{d\kappa_1}{dt} = -\kappa_1 + F_1(\kappa_1, \dots, \kappa_r, u)$$

...

$$\frac{d\kappa_r}{dt} = -\kappa_r + F_r(\kappa_1, \dots, \kappa_r, u)$$

→ dynamical system

Effective circuit

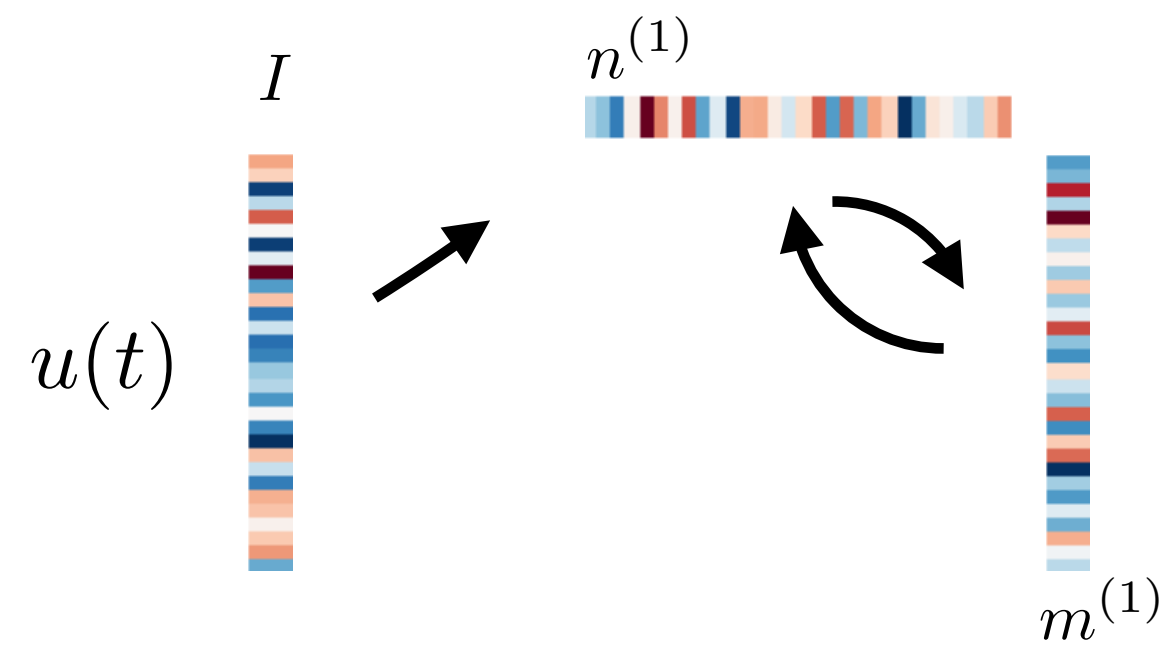


[Dubreuil*, Valente* et al, 2022]

[Beiran et al, 2021]

Example: rank 1, linear

Network

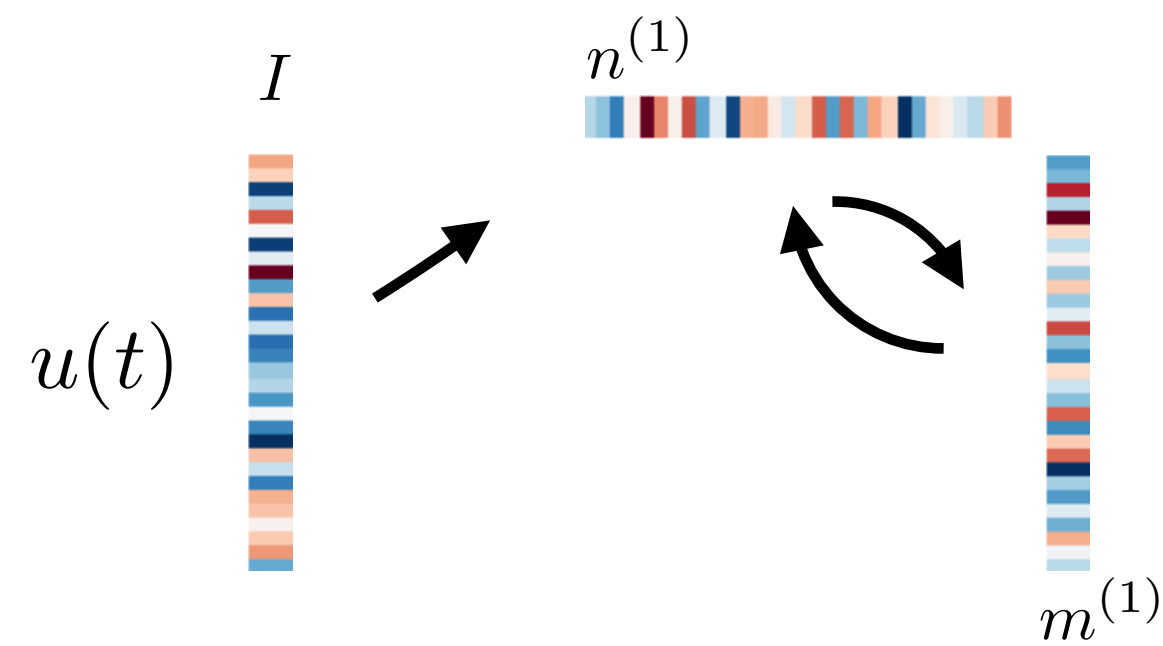


Population activity

$$\frac{dx_i}{dt} = -x_i + \sum_j \underbrace{\frac{m_i^{(1)} n_j^{(1)}}{N}}_{J_{ij}} x_j + u(t) I_i \quad (1)$$

Example: rank 1, linear

Network



Population activity

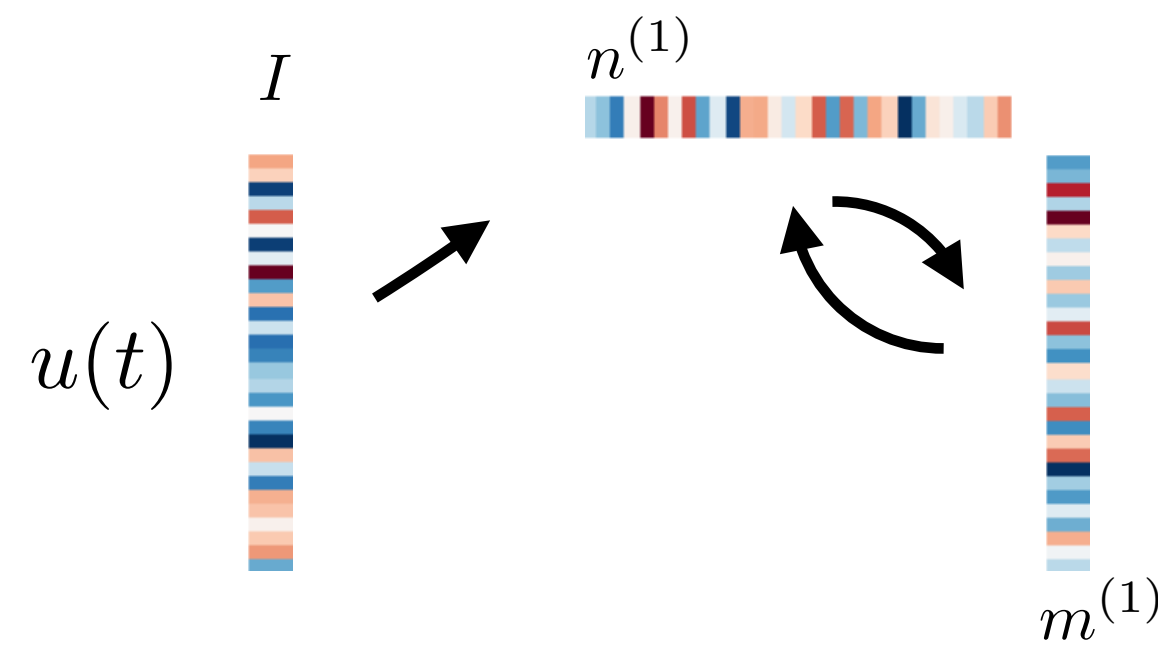
$$\frac{dx_i}{dt} = -x_i + m_i^{(1)} \sum_j \frac{n_j^{(1)}}{N} x_j + u(t) I_i \quad (1)$$

State space

$$x_i = \kappa_1(t) m_i^{(1)} + v(t) I_i$$

Example: rank 1, linear

Network



Population activity

$$\frac{dx_i}{dt} = -x_i + m_i^{(1)} \sum_j \frac{n_j^{(1)}}{N} x_j + u(t) I_i \quad (1)$$

State space

$$x_i = \kappa_1(t) m_i^{(1)} + v(t) I_i$$

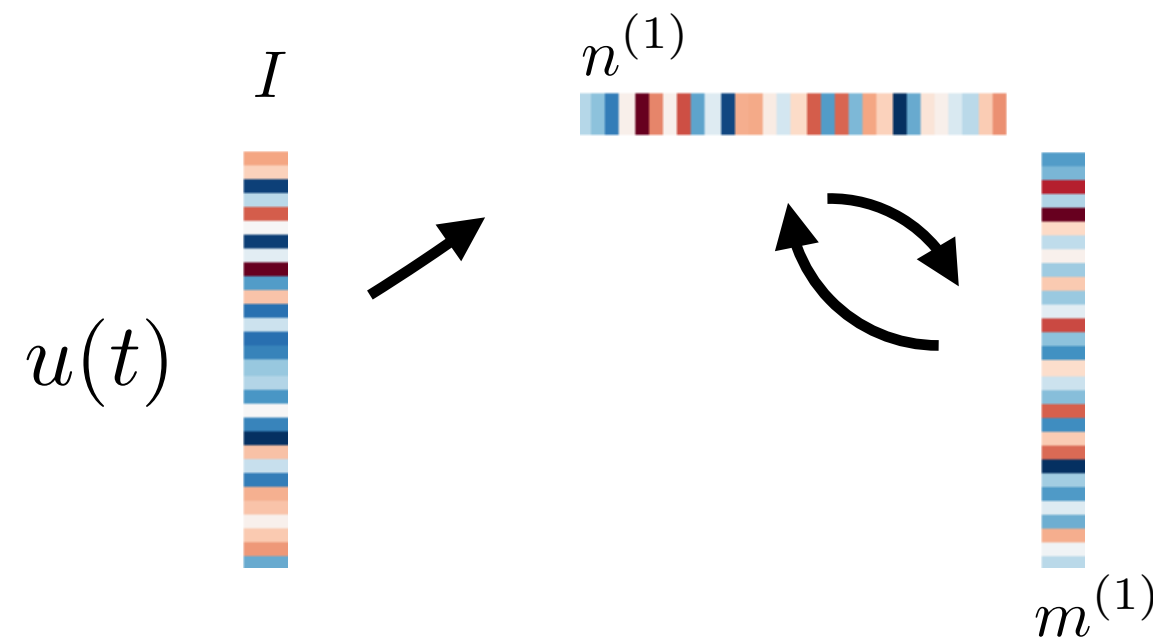
Latent dynamics

[project (1) on I]

$$\frac{dv}{dt} = -v + u(t)$$

Example: rank 1, linear

Network



Population activity

$$\frac{dx_i}{dt} = -x_i + m_i^{(1)} \sum_j \frac{n_j^{(1)}}{N} x_j + u(t) I_i \quad (1)$$

State space

$$x_i = \kappa_1(t) m_i^{(1)} + v(t) I_i$$

Latent dynamics

[project (1) on I]

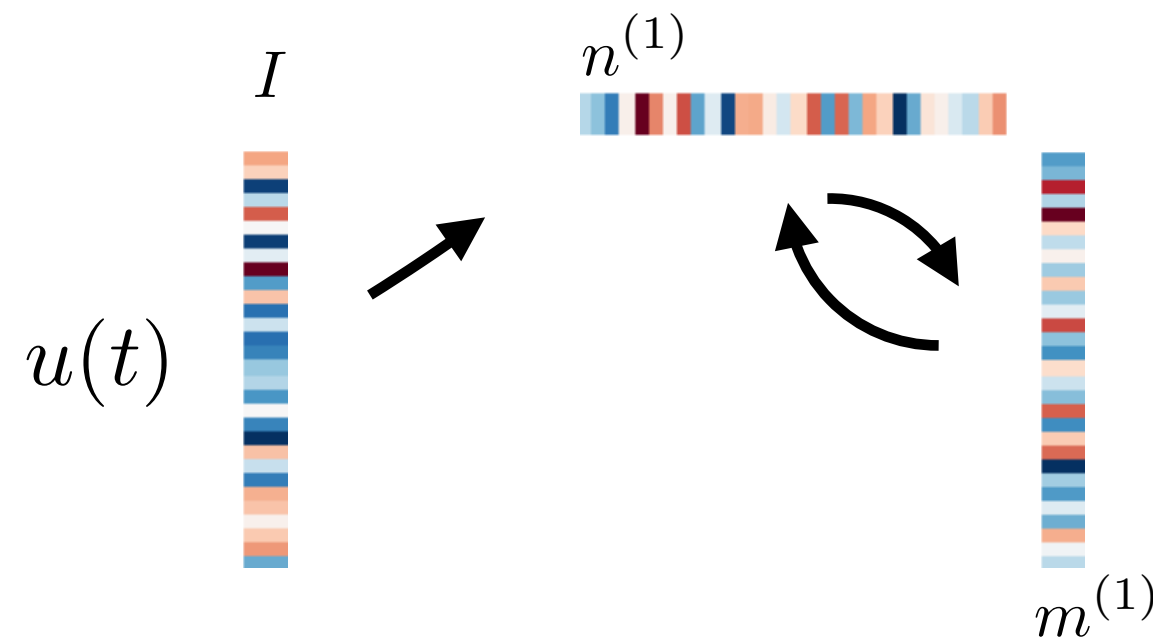
$$\frac{dv}{dt} = -v + u(t)$$

[project (1) on m]

$$\frac{d\kappa_1}{dt} = -\kappa_1 + \frac{1}{N} \sum_{j=1}^N n_j^{(1)} x_j$$

Example: rank 1, linear

Network



Population activity

$$\frac{dx_i}{dt} = -x_i + m_i^{(1)} \sum_j \frac{n_j^{(1)}}{N} x_j + u(t) I_i \quad (1)$$

State space

$$x_i = \kappa_1(t) m_i^{(1)} + v(t) I_i$$

Latent dynamics

[project (1) on I]

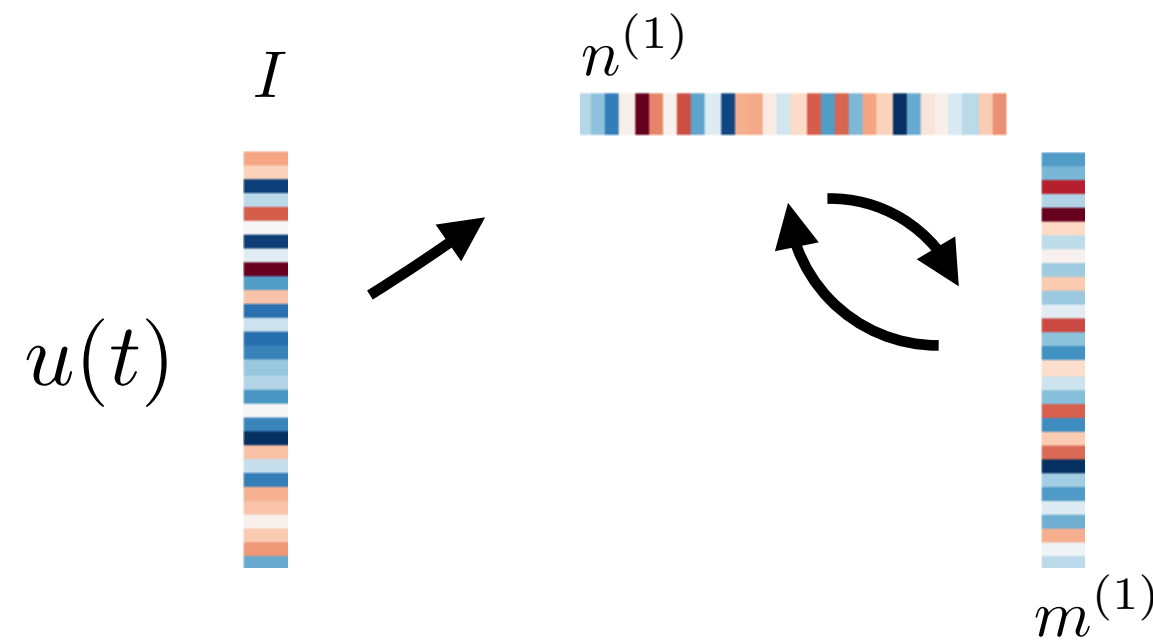
$$\frac{dv}{dt} = -v + u(t)$$

[project (1) on m]

$$\begin{aligned} \frac{d\kappa_1}{dt} &= -\kappa_1 + \frac{1}{N} \sum_{j=1}^N n_j^{(1)} x_j \\ &= -\kappa_1 + \frac{1}{N} \sum_{j=1}^N n_j^{(1)} m_j^{(1)} \kappa_1 + \frac{1}{N} \sum_{j=1}^N n_j^{(1)} I_j v(t) \end{aligned}$$

Example: rank 1, linear

Network



Population activity

$$\frac{dx_i}{dt} = -x_i + m_i^{(1)} \sum_j \frac{n_j^{(1)}}{N} x_j + u(t) I_i \quad (1)$$

State space

$$x_i = \kappa_1(t) m_i^{(1)} + v(t) I_i$$

Latent dynamics

[project (1) on I]

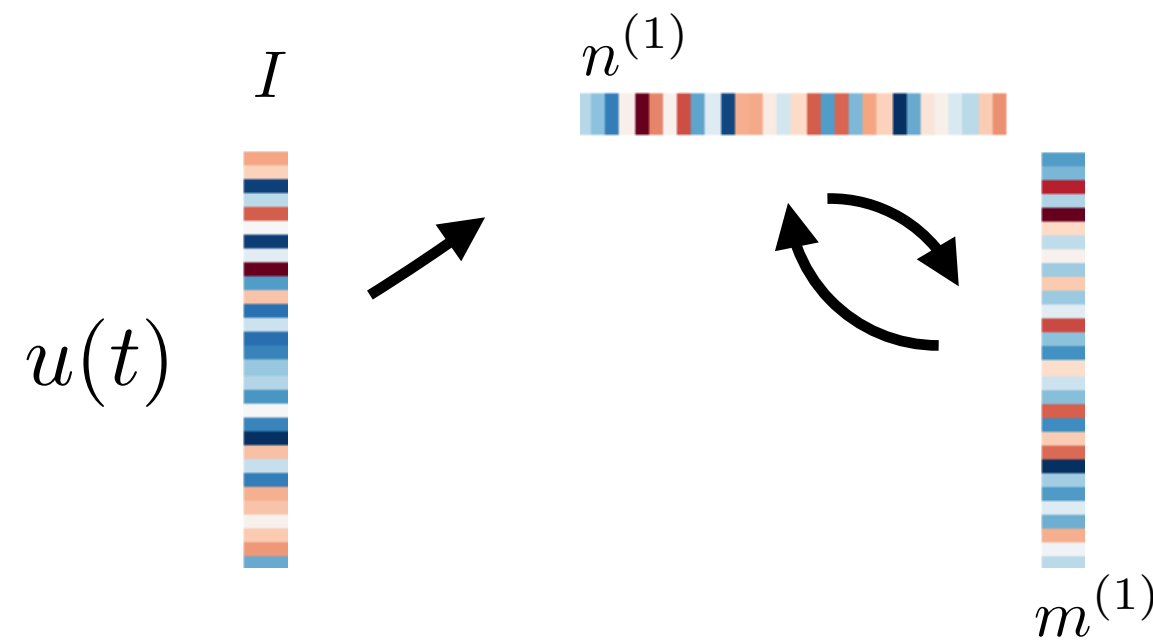
$$\frac{dv}{dt} = -v + u(t)$$

[project (1) on m]

$$\begin{aligned} \frac{d\kappa_1}{dt} &= -\kappa_1 + \frac{1}{N} \sum_{j=1}^N n_j^{(1)} x_j \\ &= -\kappa_1 + \underbrace{\frac{1}{N} \sum_{j=1}^N n_j^{(1)} m_j^{(1)}}_{\sigma_{n^{(1)} m^{(1)}}} \kappa_1 + \underbrace{\frac{1}{N} \sum_{j=1}^N n_j^{(1)} I_j}_{\sigma_{n^{(1)} I}} v(t) \end{aligned}$$

Example: rank 1, linear

Network



Population activity

$$\frac{dx_i}{dt} = -x_i + m_i^{(1)} \sum_j \frac{n_j^{(1)}}{N} x_j + u(t) I_i \quad (1)$$

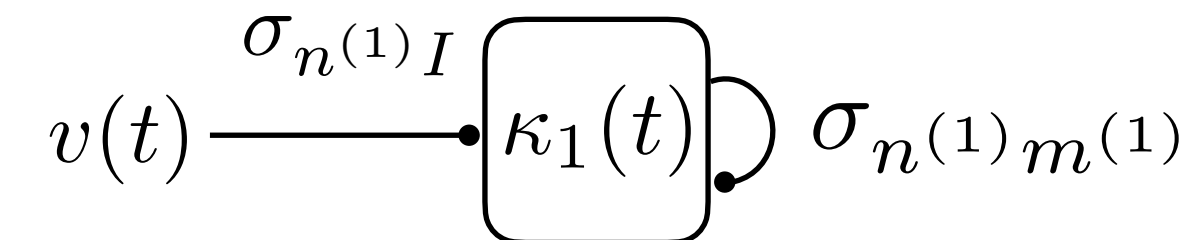
State space

$$x_i = \kappa_1(t) m_i^{(1)} + v(t) I_i$$

Latent dynamics

$$\frac{d\kappa_1}{dt} = -\kappa_1 + \underbrace{\sigma_{n^{(1)} m^{(1)}}}_{\text{red bar}} \kappa_1 + \underbrace{\sigma_{n^{(1)} I}}_{\text{red bar}} v(t)$$

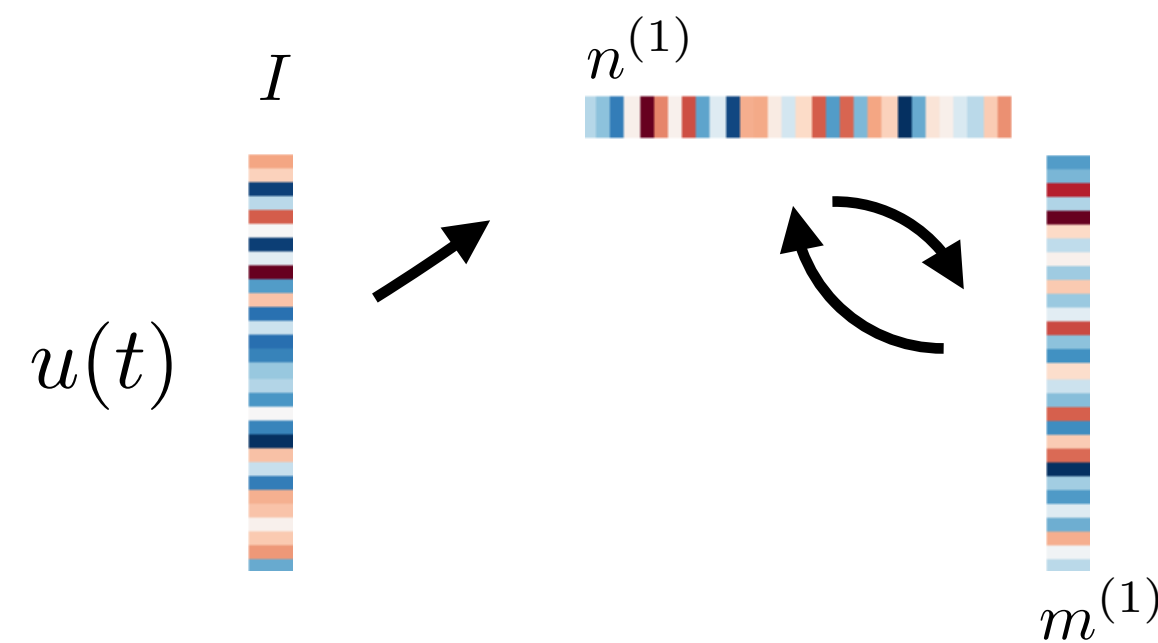
Effective circuit



➡ **Effective couplings:** overlaps/correlations
between connectivity vectors

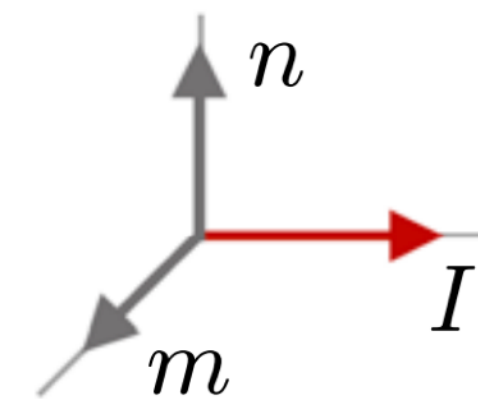
Example: rank 1, linear

Network

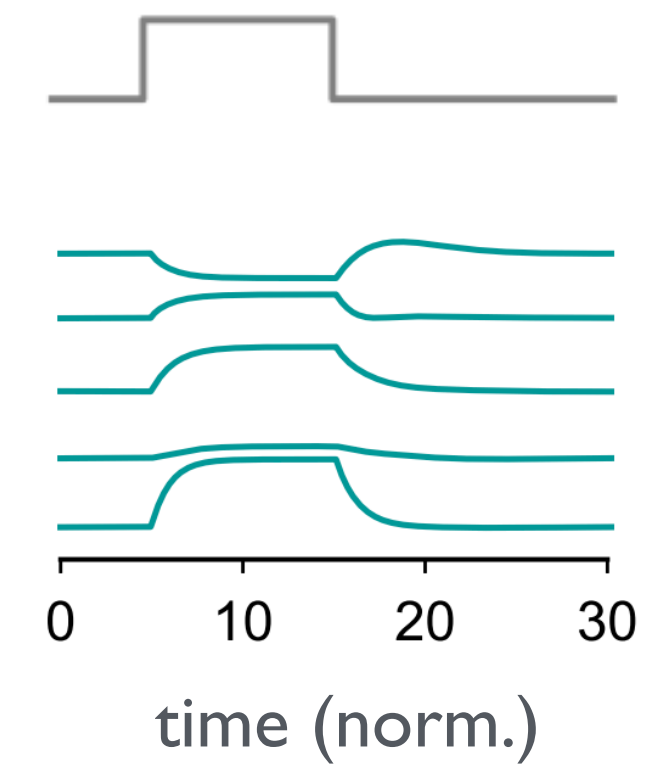


Population activity

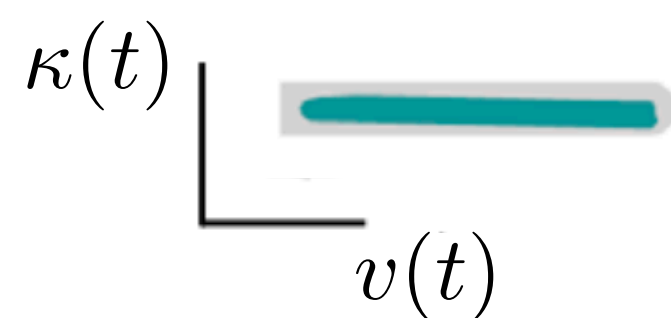
$$x_i = \cancel{\kappa_1 m_i^{(1)}} + I_i u(t)$$



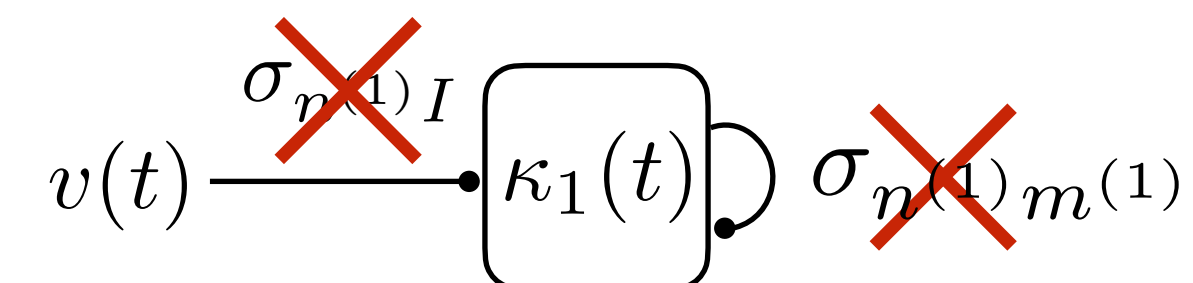
Input
Activation x_i



Latent dynamics



Effective circuit

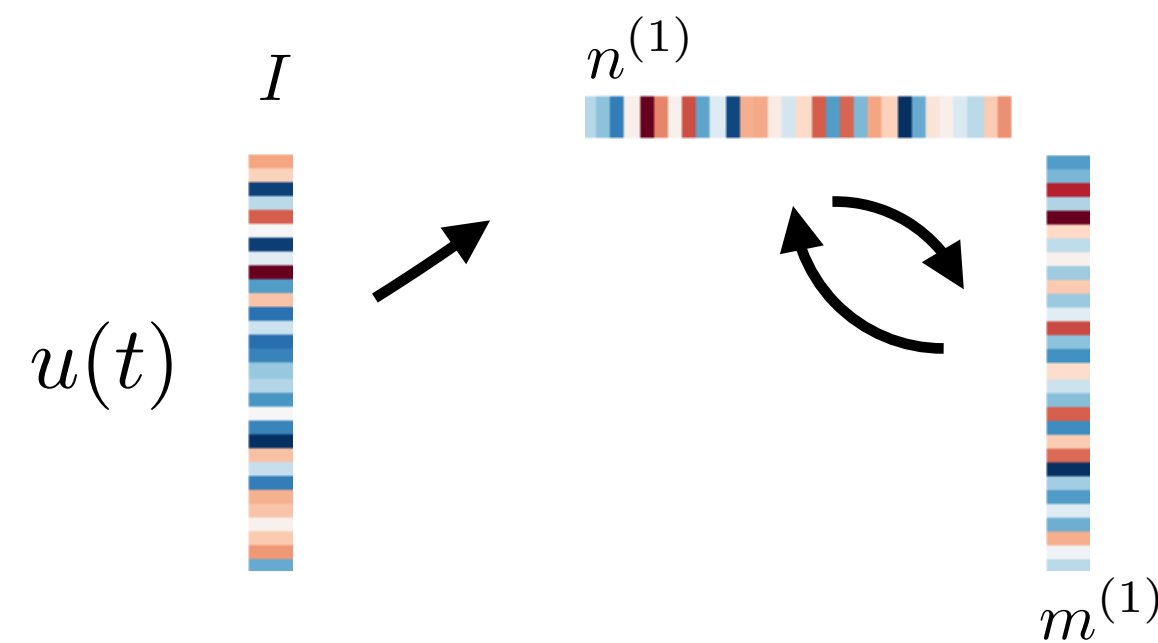


→ network replicates inputs

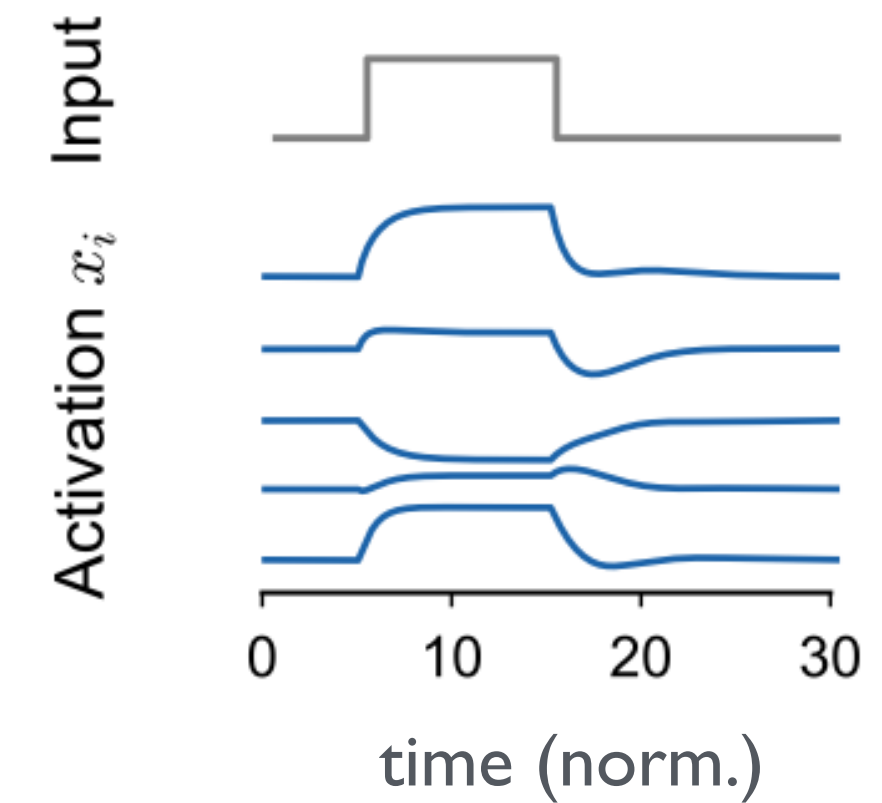
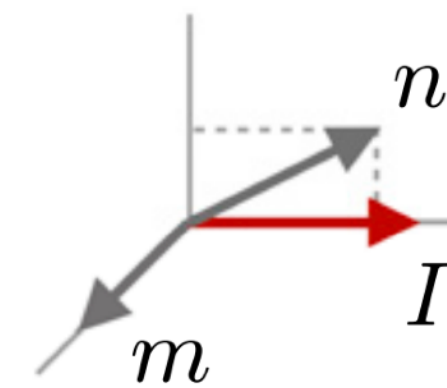
[Mastrogiuseppe & SO 2018]

Example: rank 1, linear

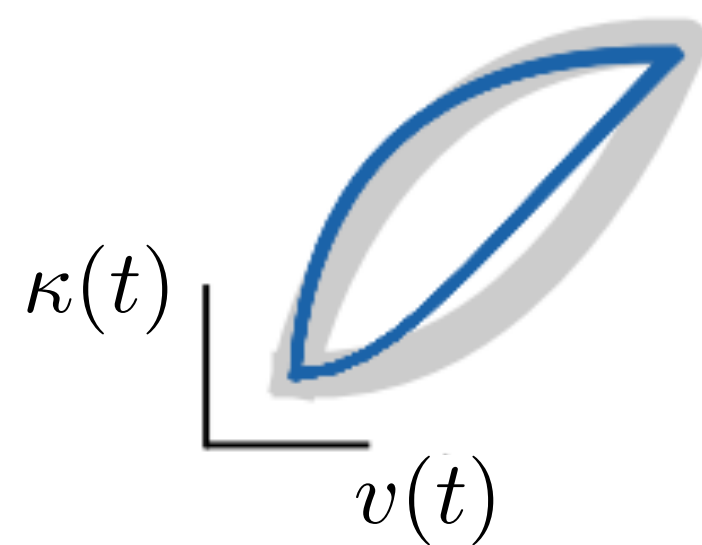
Network



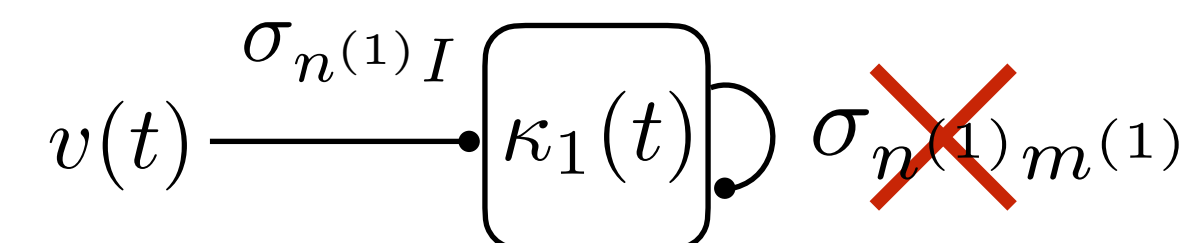
Population activity $x_i = \kappa_1 m_i^{(1)} + I_i u(t)$



Latent dynamics



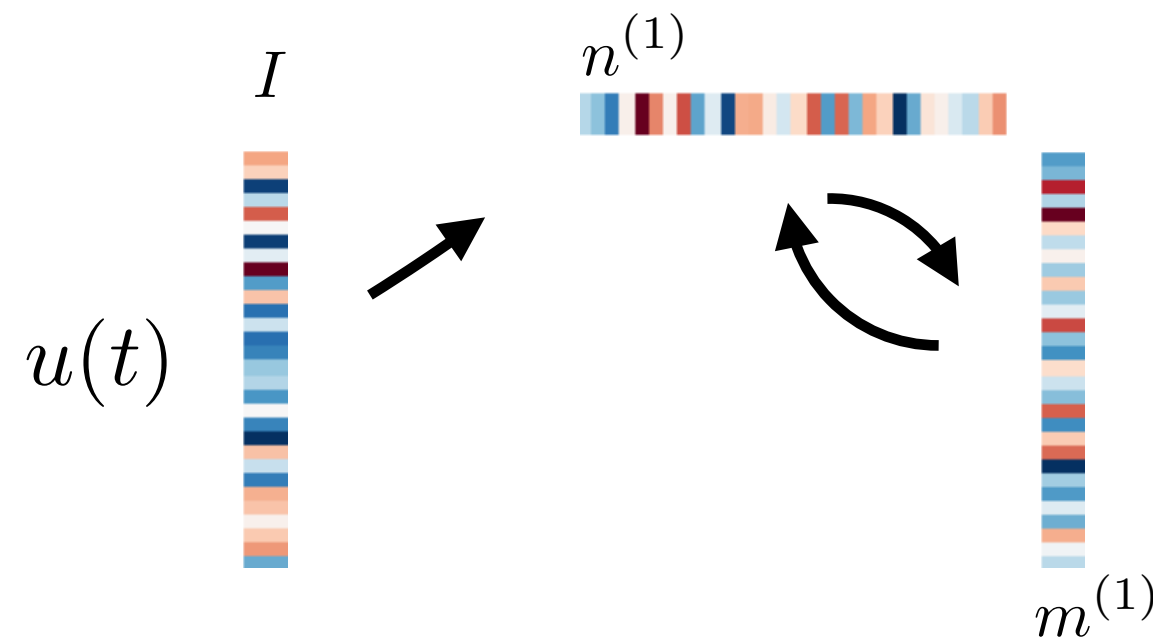
Effective circuit



→ Recurrent dynamics increase dimensionality [Mastrogiuseppe & SO 2018]

Example: rank 1, non-linear

Network



Population activity

$$\frac{dx_i}{dt} = -x_i + m_i^{(1)} \sum_j \frac{n_j^{(1)}}{N} \phi(x_j) + u(t)I_i \quad (1)$$

State space

$$x_i = \kappa_1(t)m_i^{(1)} + v(t)I_i$$

Latent dynamics

[project (1) on I]

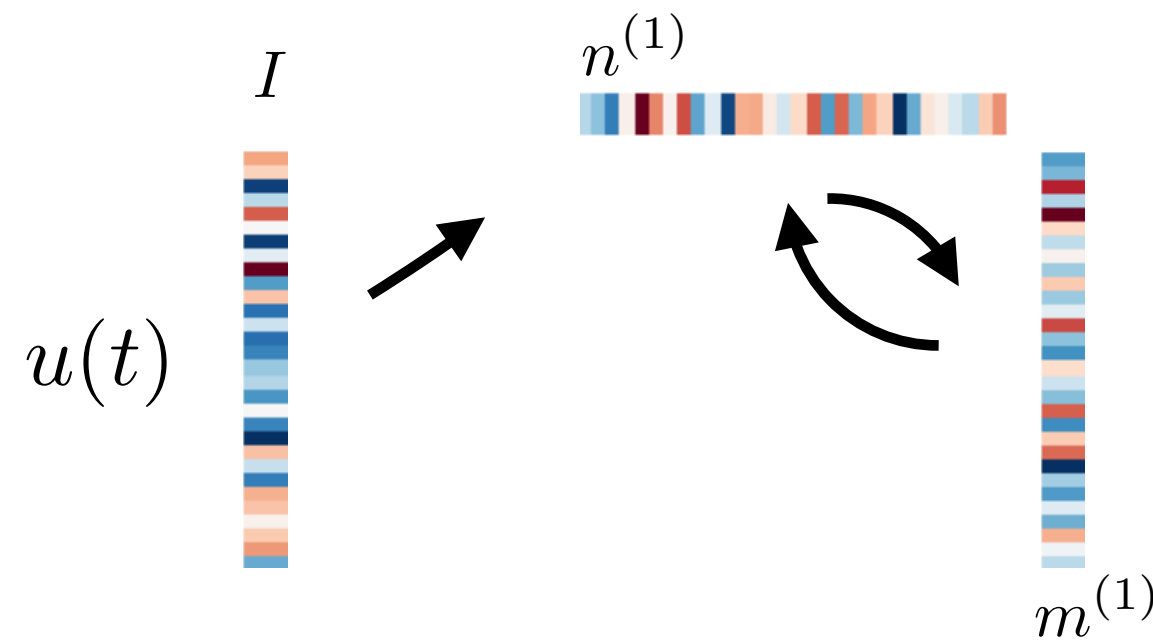
$$\frac{dv}{dt} = -v + u(t)$$

[project (1) on m]

$$\frac{d\kappa_1}{dt} = -\kappa_1 + \frac{1}{N} \sum_j n_j^{(1)} \phi(x_j)$$

Example: rank 1, non-linear

Network



Population activity

$$\frac{dx_i}{dt} = -x_i + m_i^{(1)} \sum_j \frac{n_j^{(1)}}{N} \phi(x_j) + u(t)I_i \quad (1)$$

State space

$$x_i = \kappa_1(t)m_i^{(1)} + v(t)I_i$$

Latent dynamics

[project (1) on I]

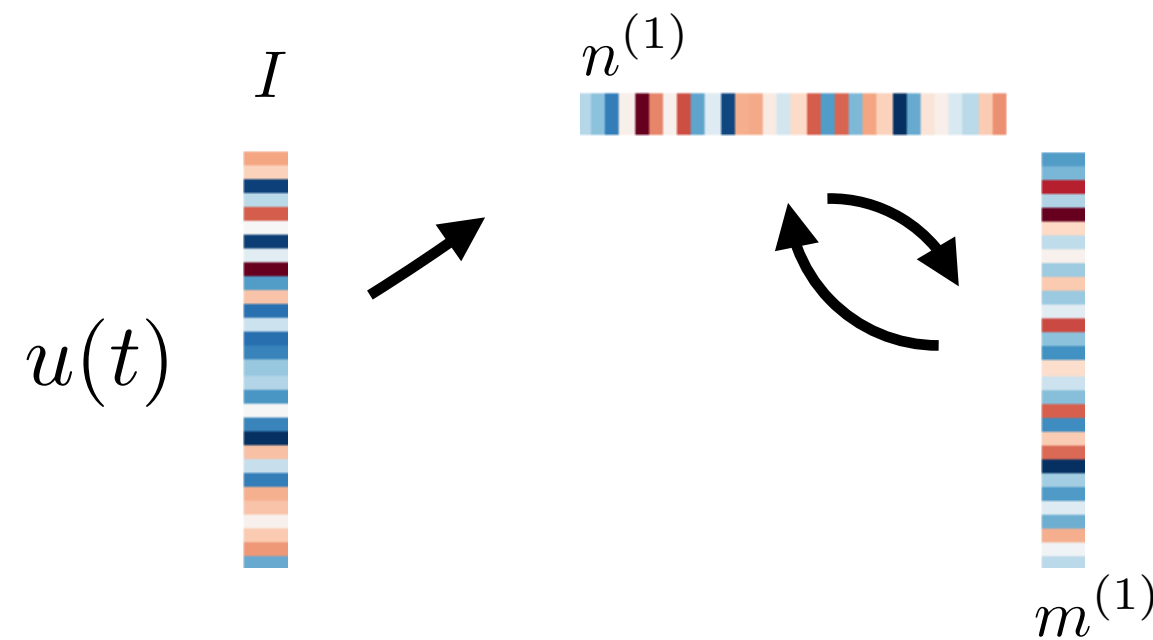
$$\frac{dv}{dt} = -v + u(t)$$

[project (1) on m]

$$\begin{aligned} \frac{d\kappa_1}{dt} &= -\kappa_1 + \frac{1}{N} \sum_j n_j^{(1)} \phi(x_j) \\ &= -\kappa_1 + \frac{1}{N} \sum_j n_j^{(1)} \phi(\kappa_1(t)m_j^{(1)} + v(t)I_j) \end{aligned}$$

Example: rank 1, non-linear

Network



Population activity

$$\frac{dx_i}{dt} = -x_i + m_i^{(1)} \sum_j \frac{n_j^{(1)}}{N} \phi(x_j) + u(t)I_i \quad (1)$$

State space

$$x_i = \kappa_1(t)m_i^{(1)} + v(t)I_i$$

Latent dynamics

[project (1) on I]

$$\frac{dv}{dt} = -v + u(t)$$

[project (1) on m]

$$\begin{aligned} \frac{d\kappa_1}{dt} &= -\kappa_1 + \frac{1}{N} \sum_j n_j^{(1)} \phi(x_j) \\ &= -\kappa_1 + \frac{1}{N} \sum_j n_j^{(1)} \phi(\kappa_1(t)m_j^{(1)} + v(t)I_j) \end{aligned}$$

$F(\kappa_1, v)$

Understanding non-linearity

$$F(\kappa, \nu) = \frac{1}{N} \sum_j n_j \phi(\kappa m_j + \nu I_j)$$

Understanding non-linearity

$$F(\kappa, \nu) = \frac{1}{N} \sum_j \overbrace{n_j}^{X_j} \underbrace{\phi(\kappa m_j + \nu l_j)}_{Y_j}$$
$$= \text{Cov}(X, Y)$$

Understanding non-linearity

$$\begin{aligned} F(\kappa, \nu) &= \frac{1}{N} \sum_j n_j \phi(\kappa m_j + \nu I_j) \\ &= \text{Cov}(n, \phi(\kappa m + \nu I)) \end{aligned}$$

Understanding non-linearity

$$F(\kappa, \nu) = \frac{1}{N} \sum_j n_j \phi(\kappa m_j + \nu I_j)$$
$$= \text{Cov}(n, \phi(\kappa m + \nu I))$$

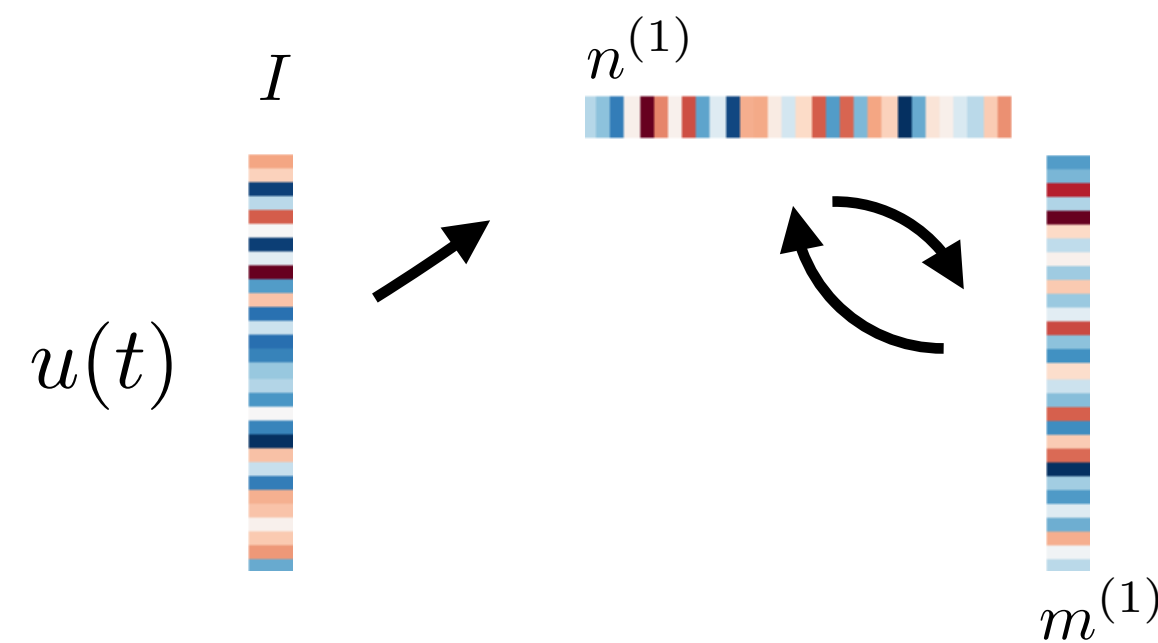
Mean-field limit:

$$n_j, m_j, I_j \sim P(n, m, I)$$
$$= \int dn dm dI P(n, m, I) n \phi(\kappa m + \nu I)$$

→ Dynamics determined by statistics of connectivity

Example: rank 1, non-linear, Gaussian iid

Network



Population activity

$$\frac{dx_i}{dt} = -x_i + m_i^{(1)} \sum_j \frac{n_j^{(1)}}{N} \phi(x_j) + u(t)I_i \quad (1)$$

State space

$$x_i = \kappa_1(t)m_i^{(1)} + v(t)I_i$$

Latent dynamics

[project (1) on I]

$$\frac{dv}{dt} = -v + u(t)$$

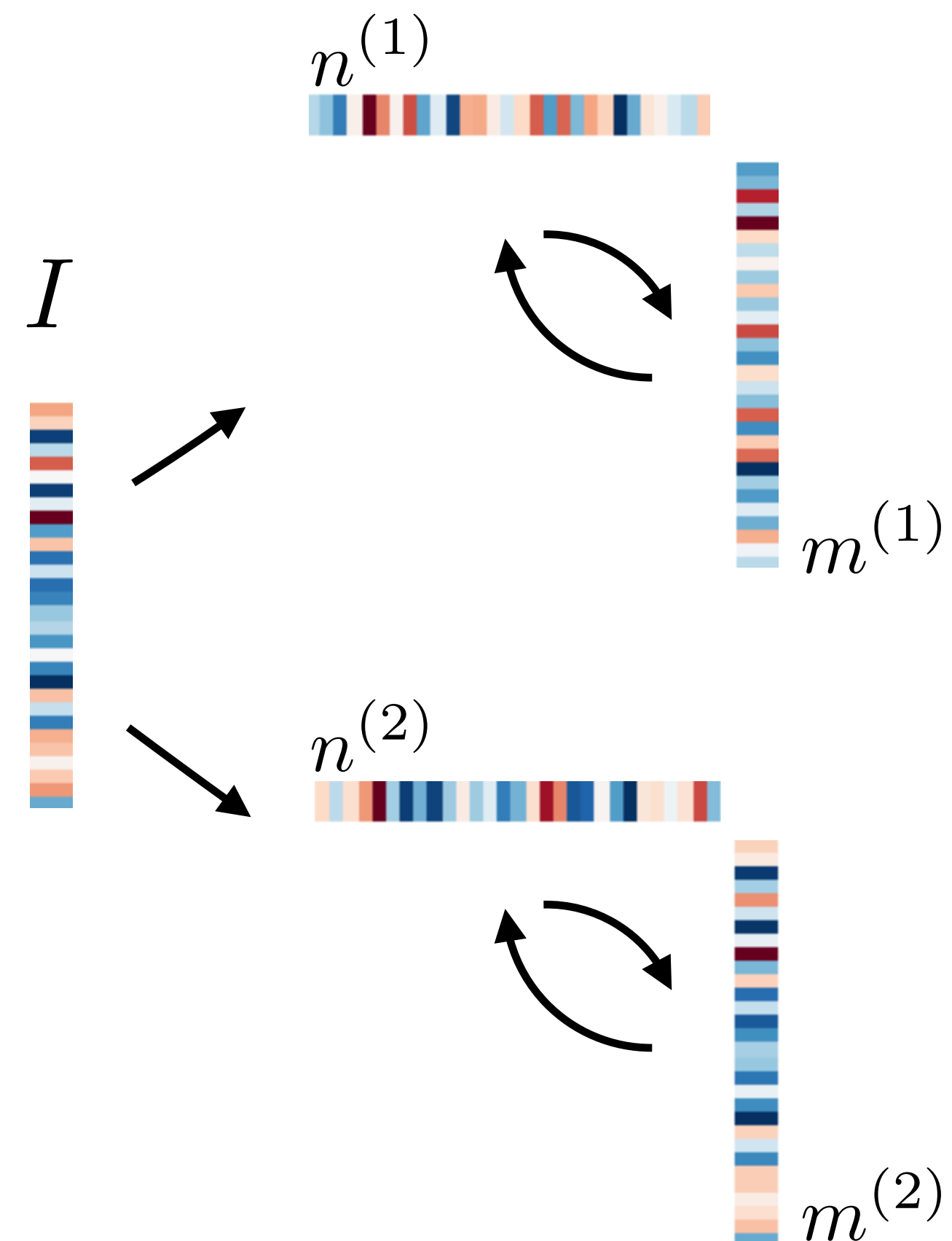
[project (1) on m]

$$\begin{aligned} \frac{d\kappa_1}{dt} &= -\kappa_1 + \frac{1}{N} \sum_j n_j^{(1)} \phi(x_j) \\ &= -\kappa_1 + \frac{1}{N} \sum_j n_j^{(1)} \phi(\kappa_1(t)m_j^{(1)} + v(t)I_j) \end{aligned}$$

$F(\kappa_1, v) = \text{Latent dynamics}$

Connectivity statistics

Input Connectivity



[Dubreuil*, Valente* et al, 2022]

[Beiran et al, 2021]

Connectivity statistics

Input

Connectivity

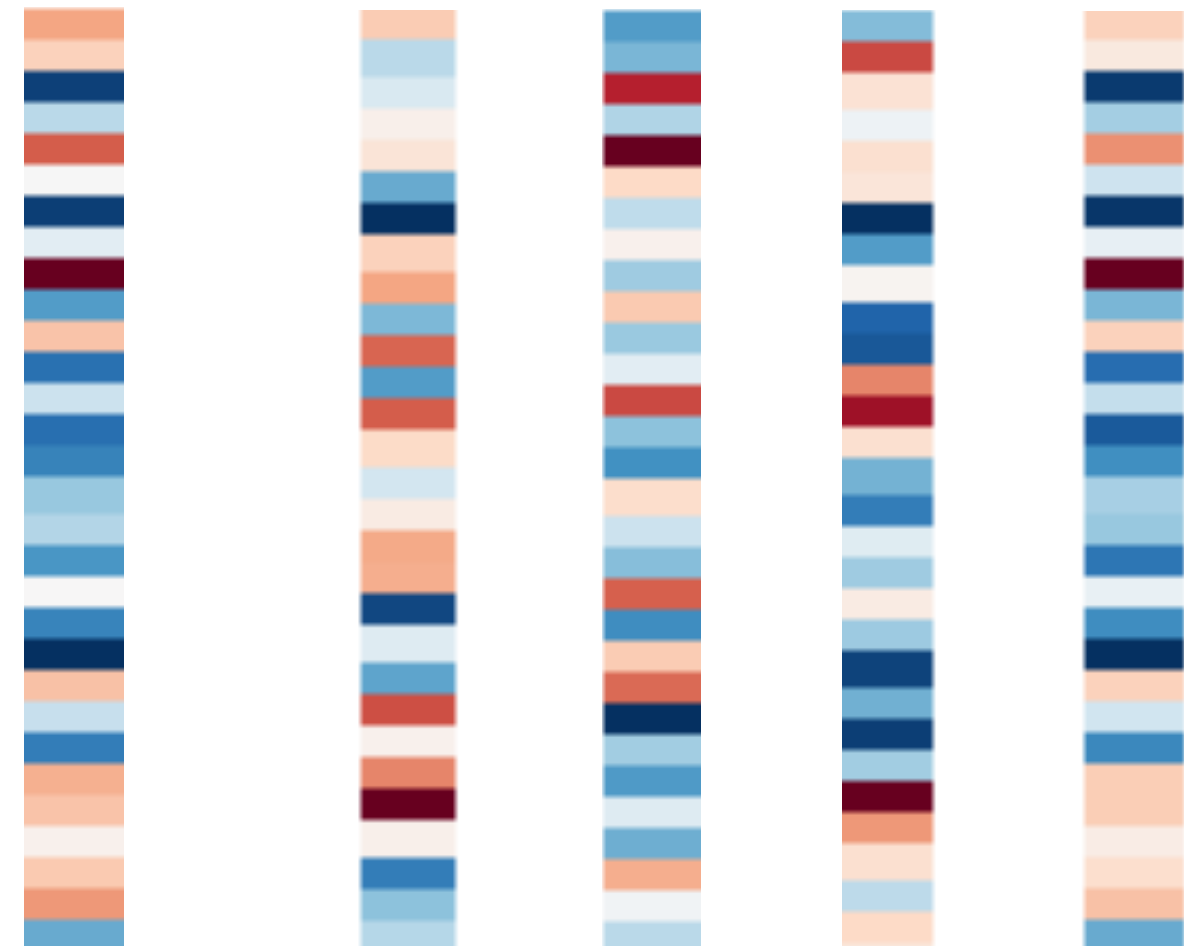
I

$n^{(1)}$

$m^{(1)}$

$n^{(2)}$

$m^{(2)}$



[Dubreuil*, Valente* et al, 2022]

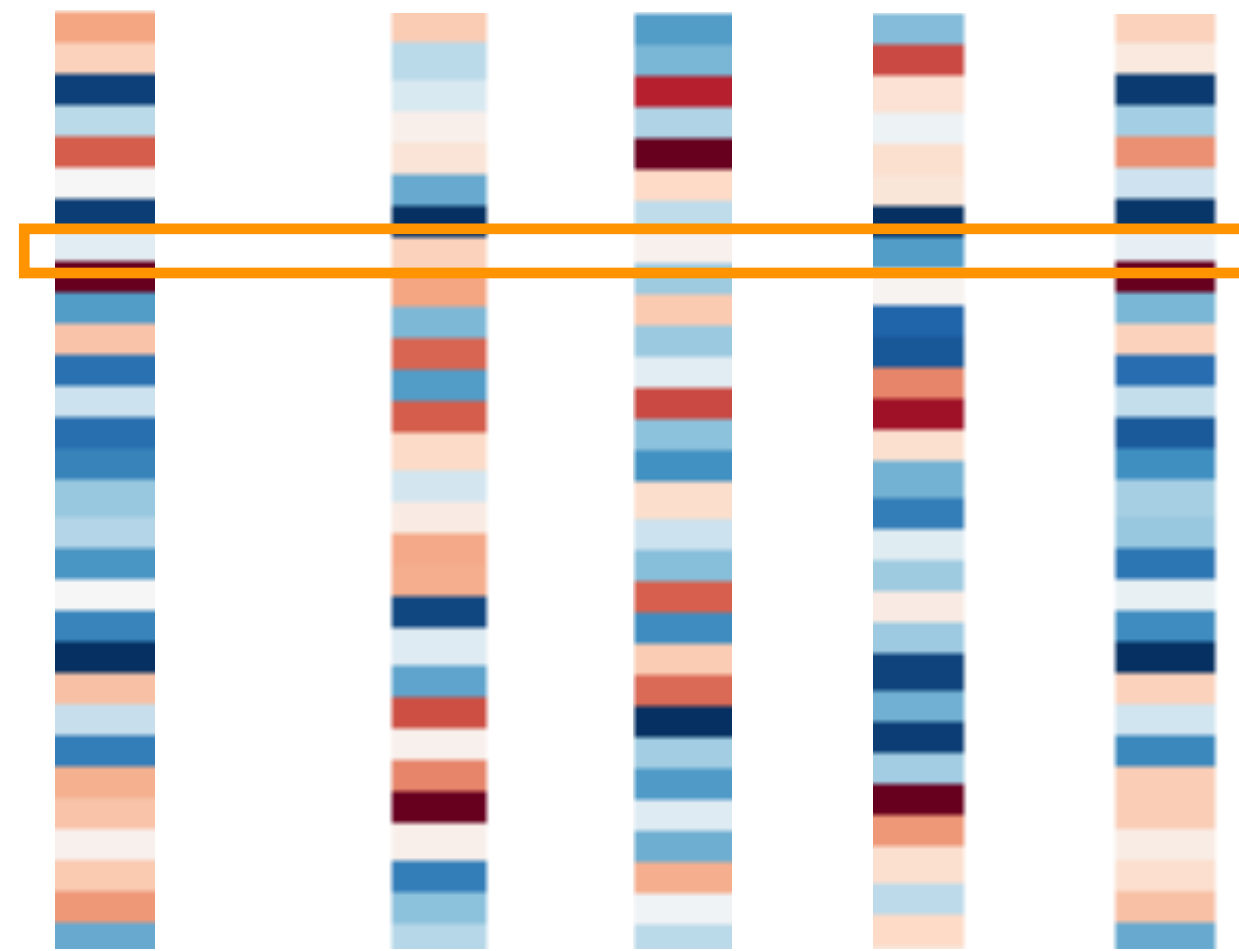
[Beiran et al, 2021]

Connectivity statistics

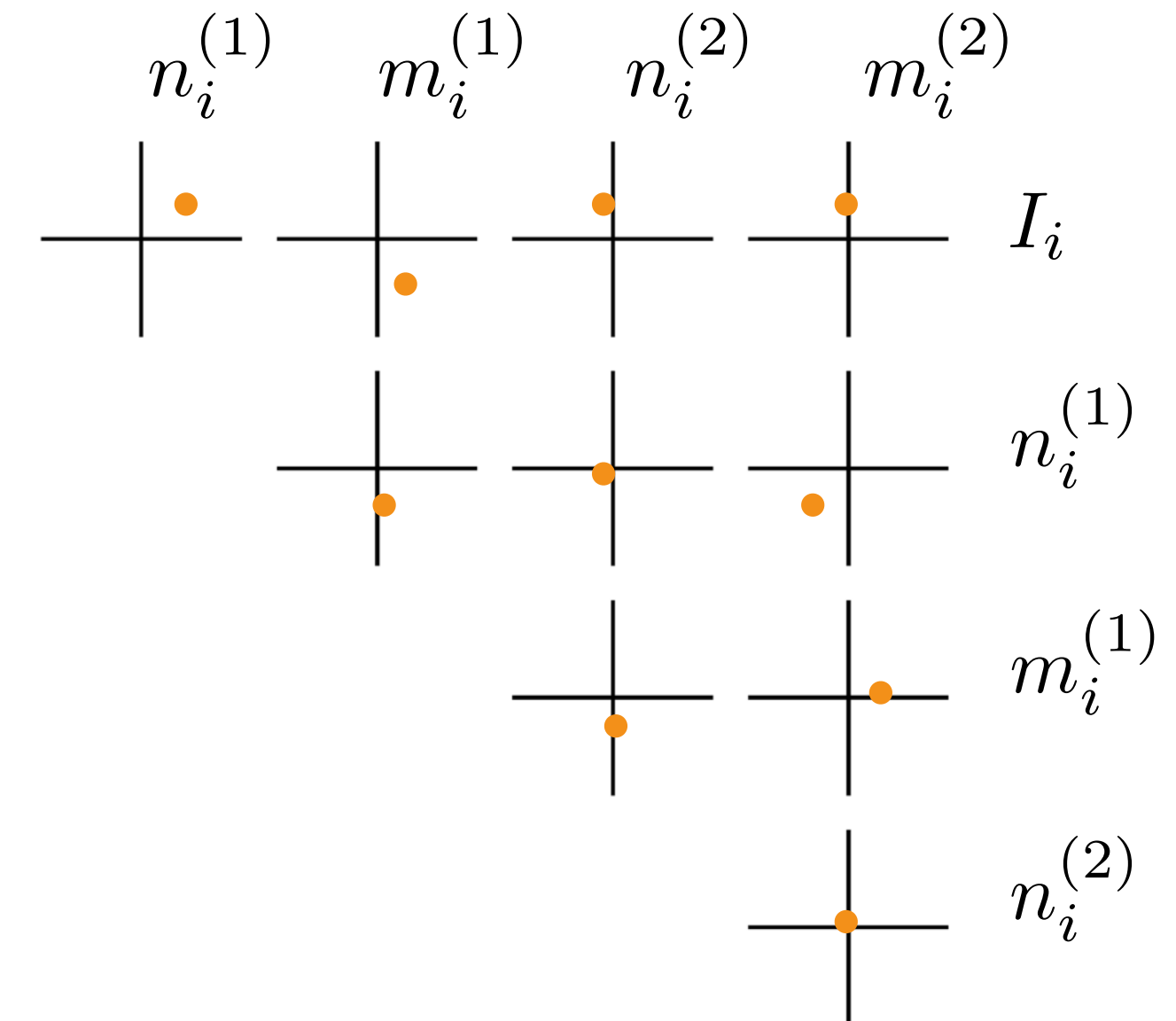
Input

Connectivity

I

$$n^{(1)}$$
$$m^{(1)}$$
$$n^{(2)}$$
$$m^{(2)}$$


Connectivity space



[Dubreuil*,Valente* et al, 2022]

[Beiran et al, 2021]

Connectivity statistics

Input

Connectivity

Connectivity space

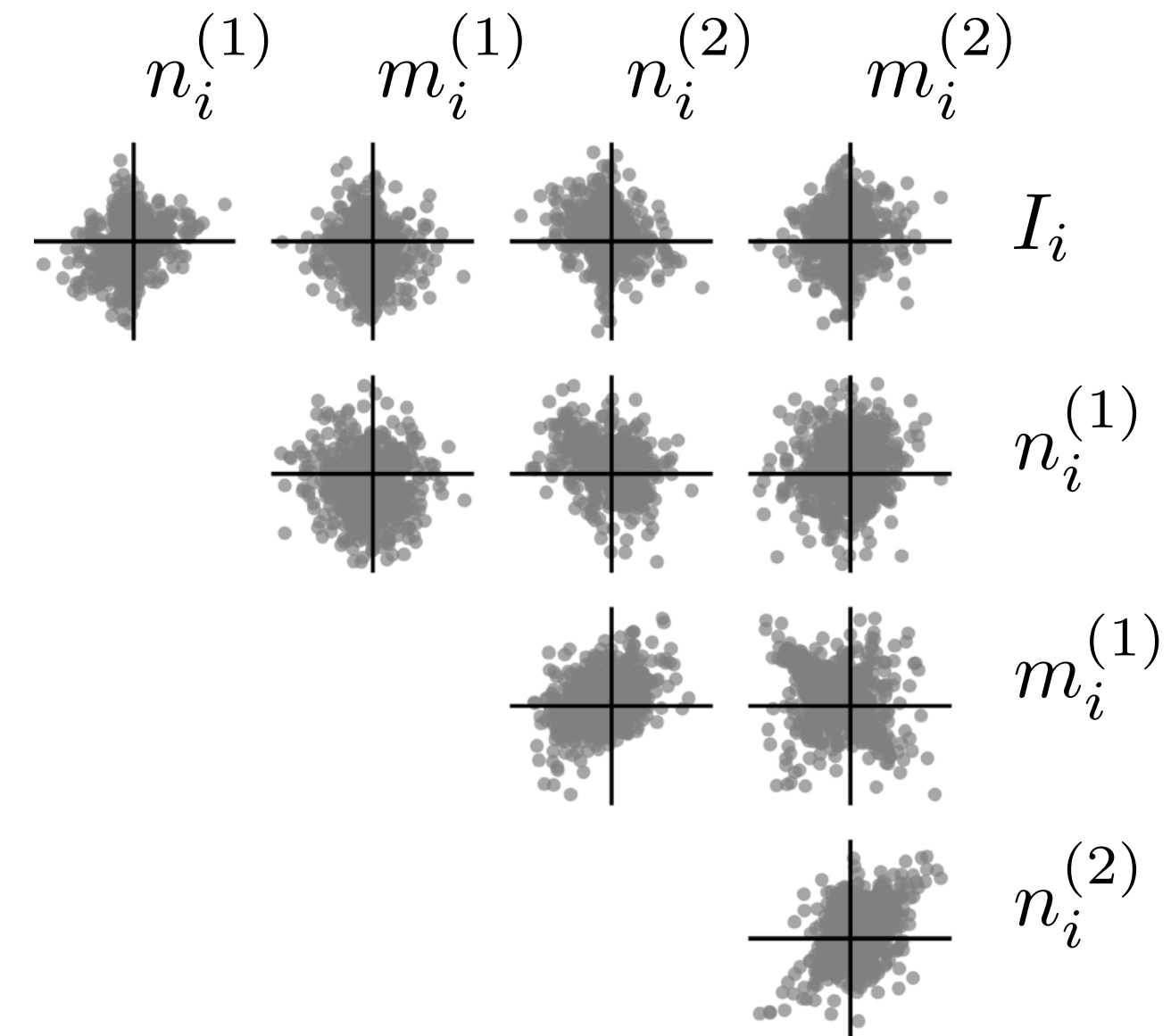
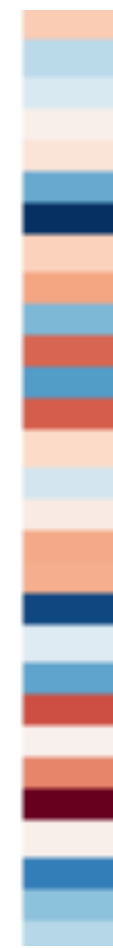
I

$n^{(1)}$

$m^{(1)}$

$n^{(2)}$

$m^{(2)}$



Distribution determines
latent dynamics

Multivariate Gaussian

Input

Connectivity

Connectivity space

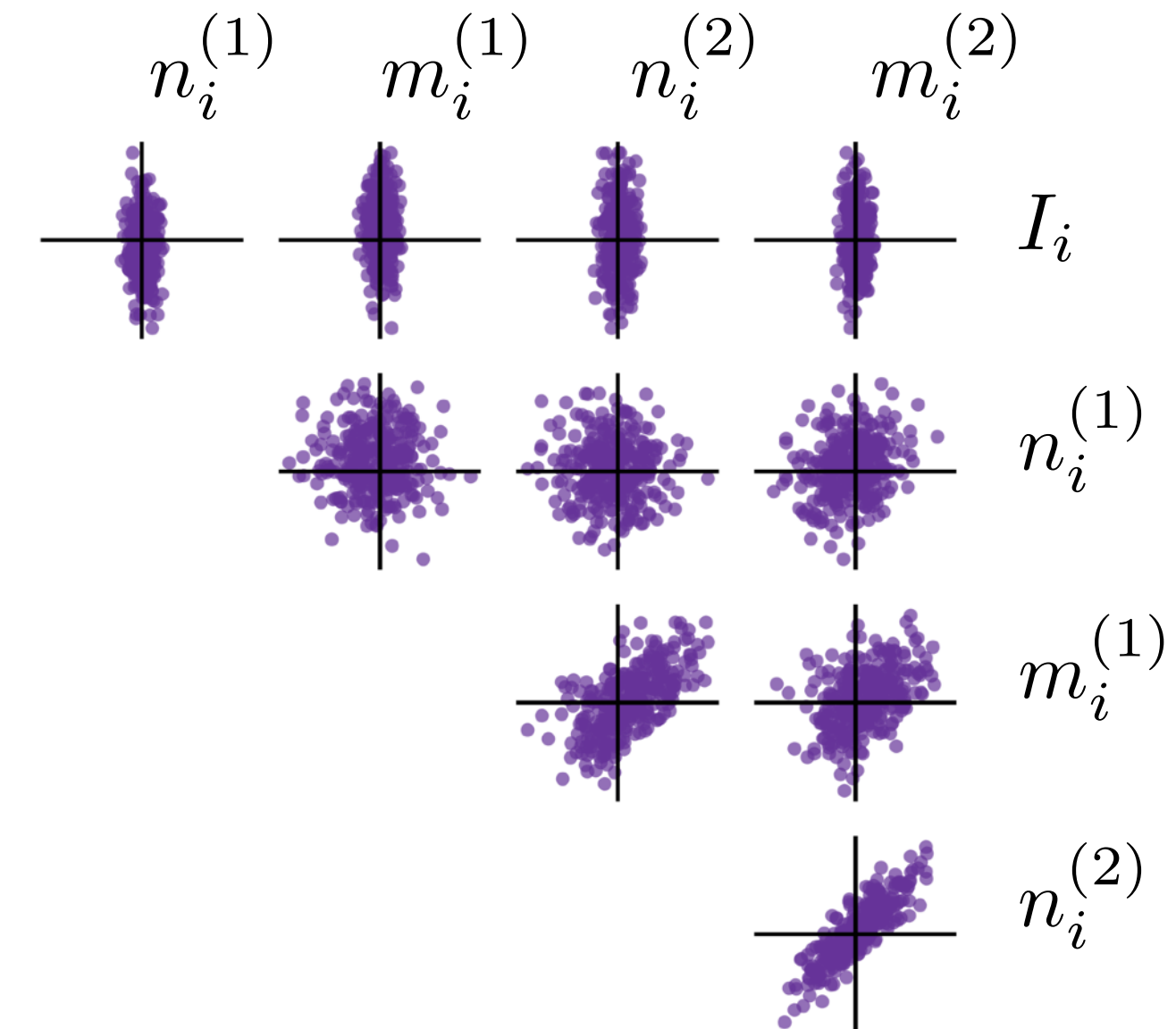
I

$n^{(1)}$

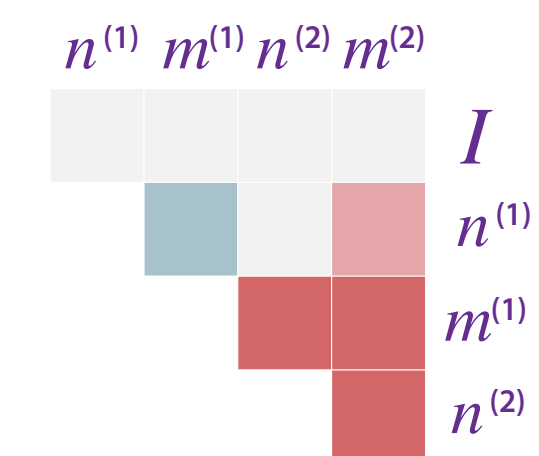
$m^{(1)}$

$n^{(2)}$

$m^{(2)}$



Covariance

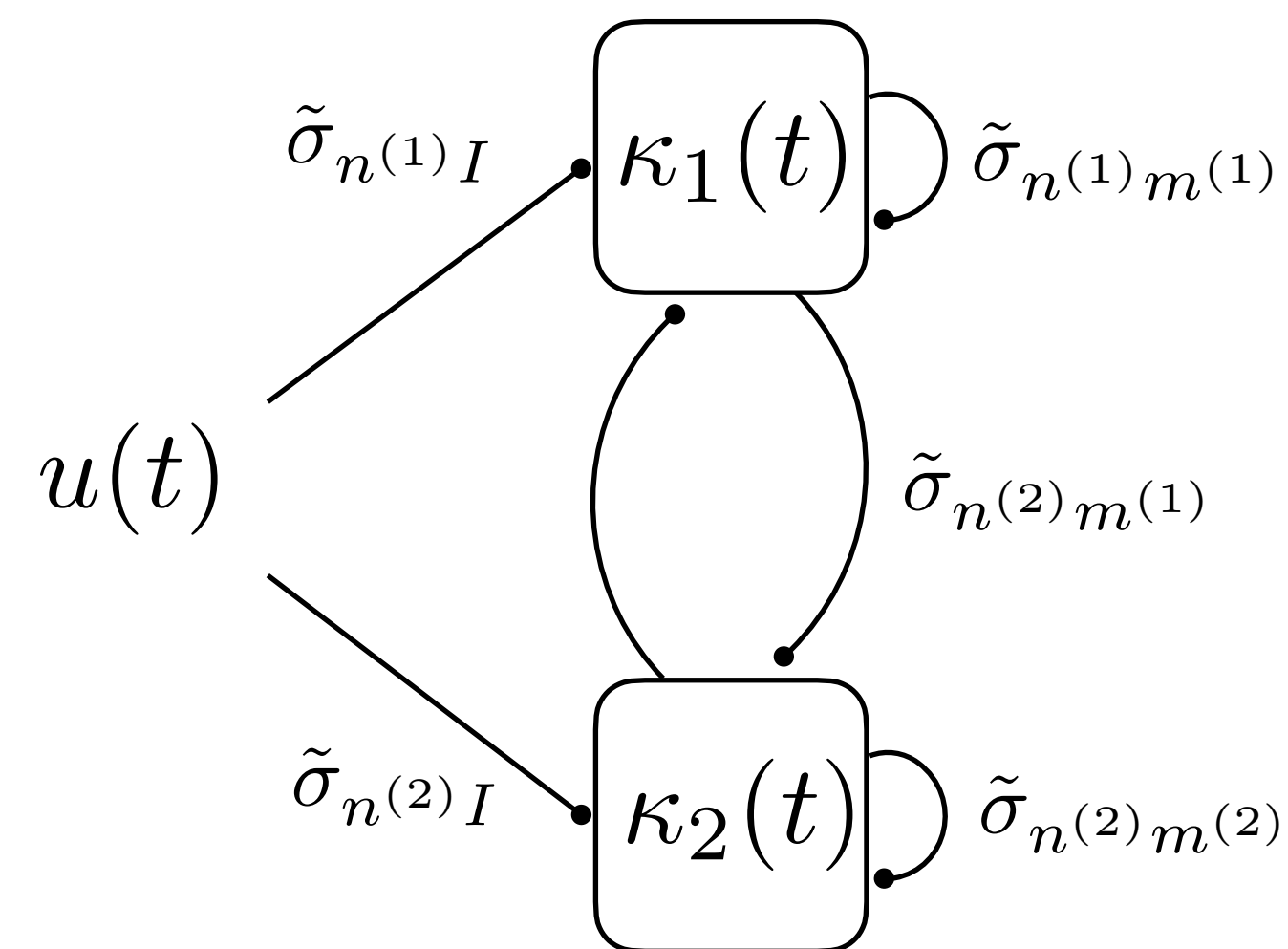


Minimal structure = maximally random

Effective circuit

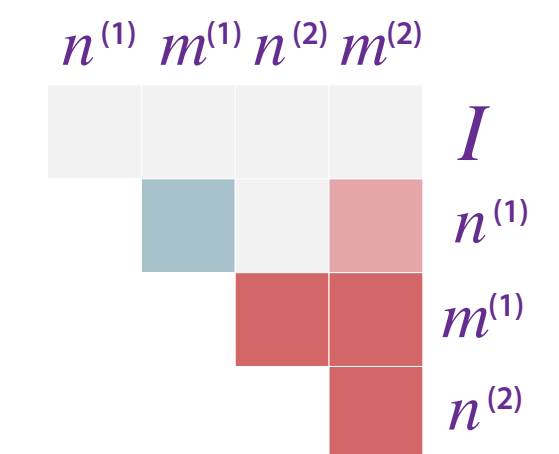
Latent dynamics

Effective couplings



$$\tilde{\sigma} = \langle \phi' \rangle \times$$

average gain



Pattern overlaps



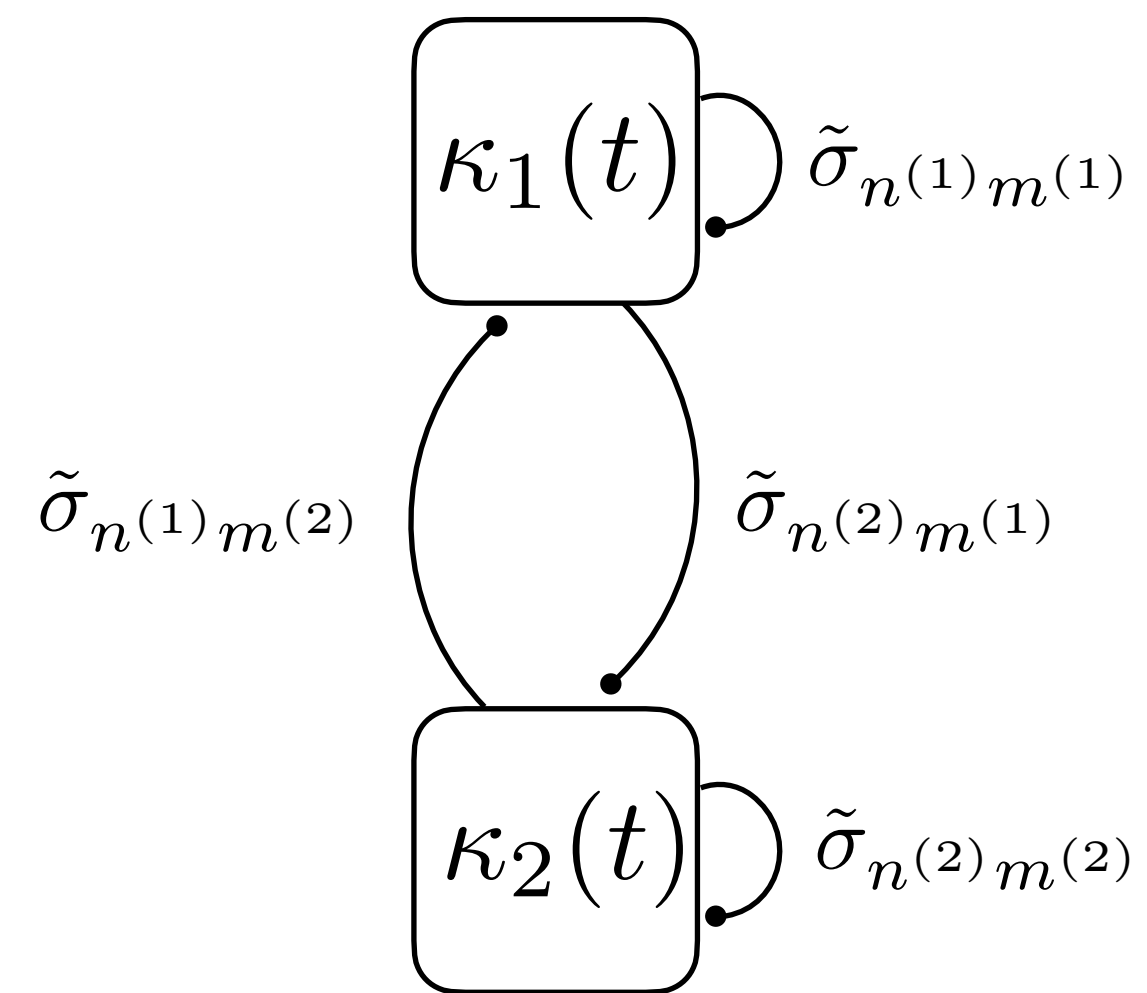
Exact non-linear description, but limited range of dynamics

[Beiran et al, 2021]

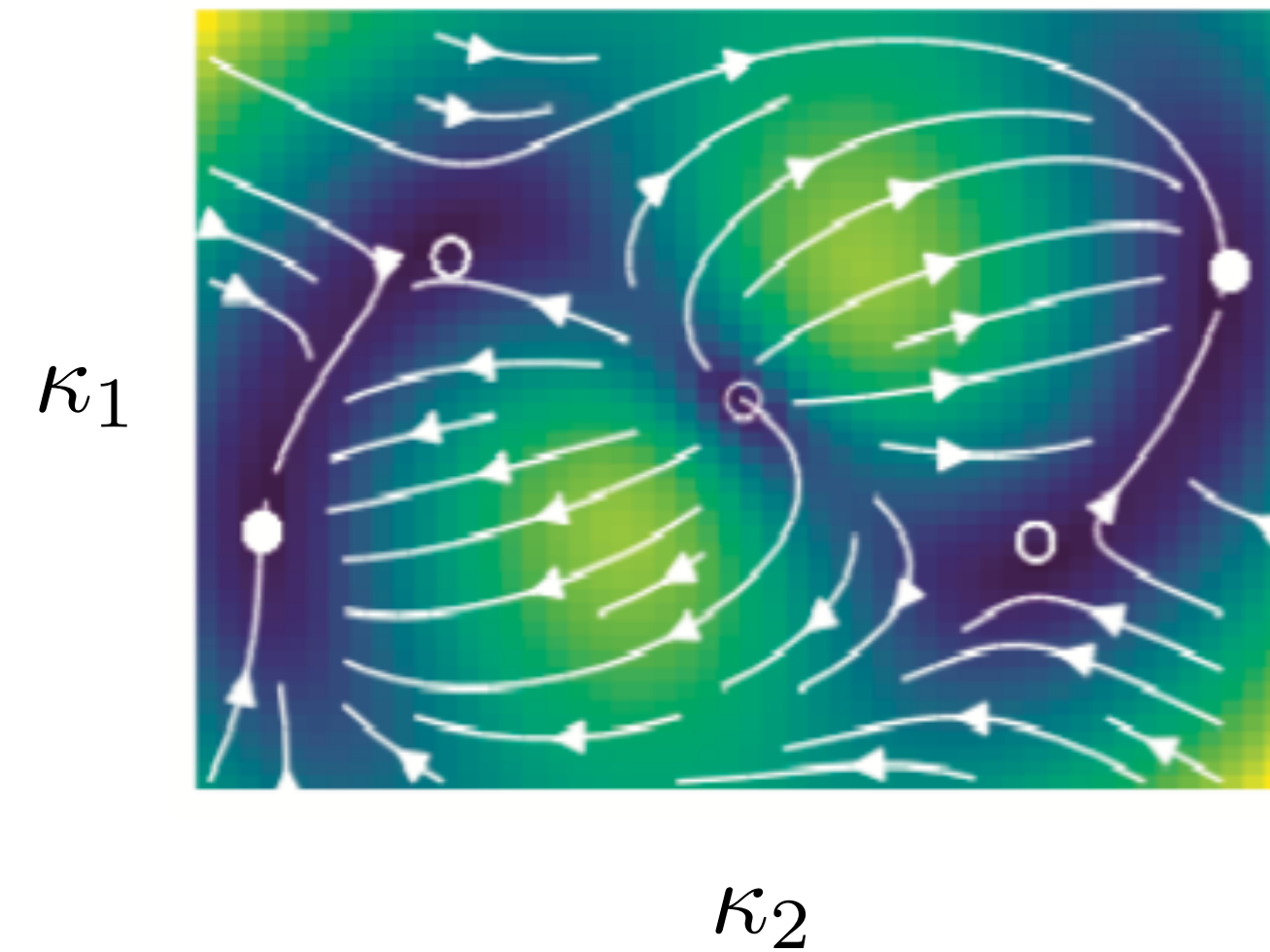
$$\begin{aligned} \frac{d\kappa_1}{dt} &= -\kappa_1 + \tilde{\sigma}_{n^{(1)}m^{(1)}} \kappa_1 + \tilde{\sigma}_{n^{(1)}m^{(2)}} \kappa_2 + \tilde{\sigma}_{n^{(1)}I} u(t) \\ \frac{d\kappa_2}{dt} &= -\kappa_2 + \tilde{\sigma}_{n^{(2)}m^{(1)}} \kappa_1 + \tilde{\sigma}_{n^{(2)}m^{(2)}} \kappa_2 + \tilde{\sigma}_{n^{(2)}I} u(t) \end{aligned}$$

Autonomous dynamics

Latent dynamics

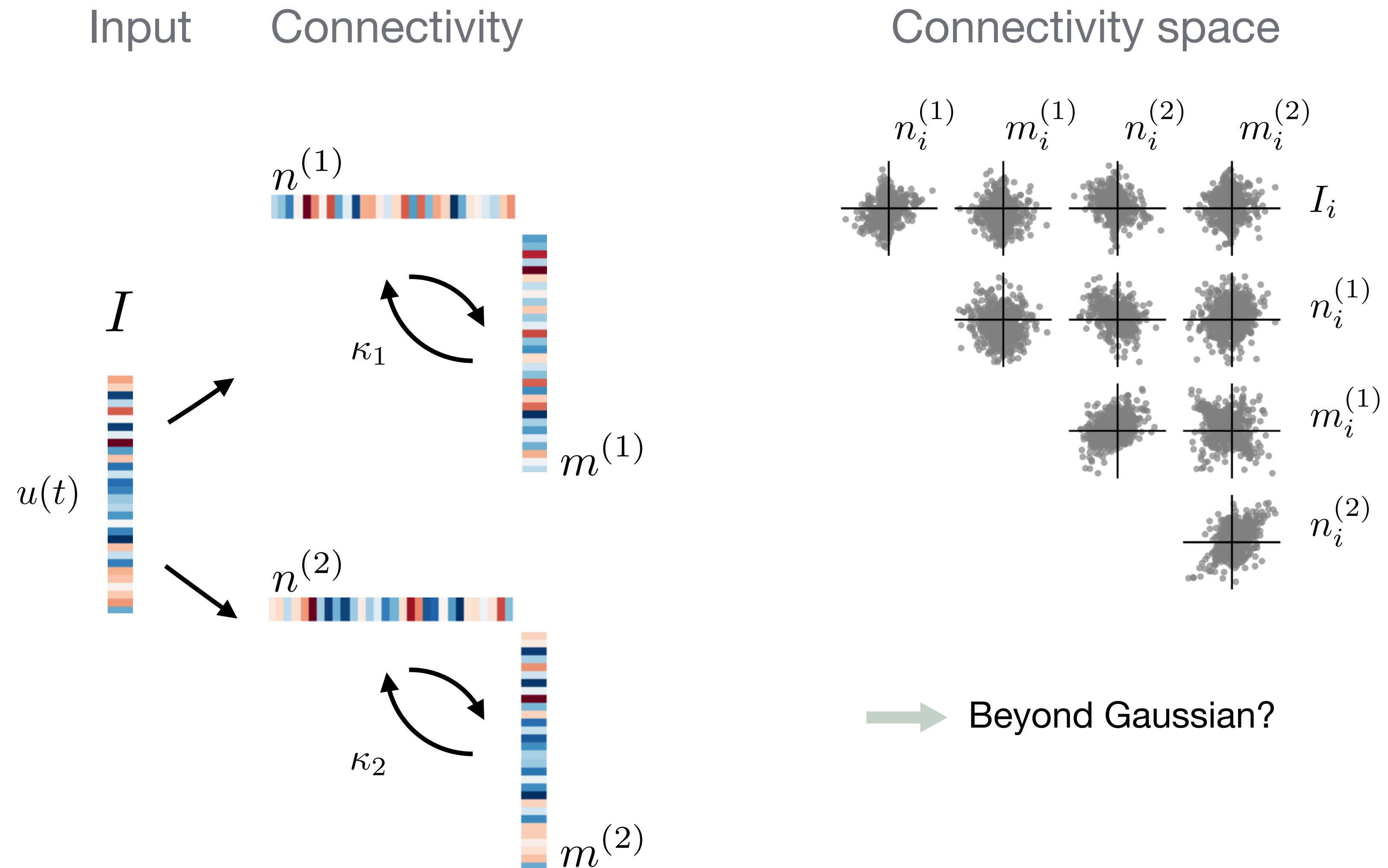


Dynamical landscape

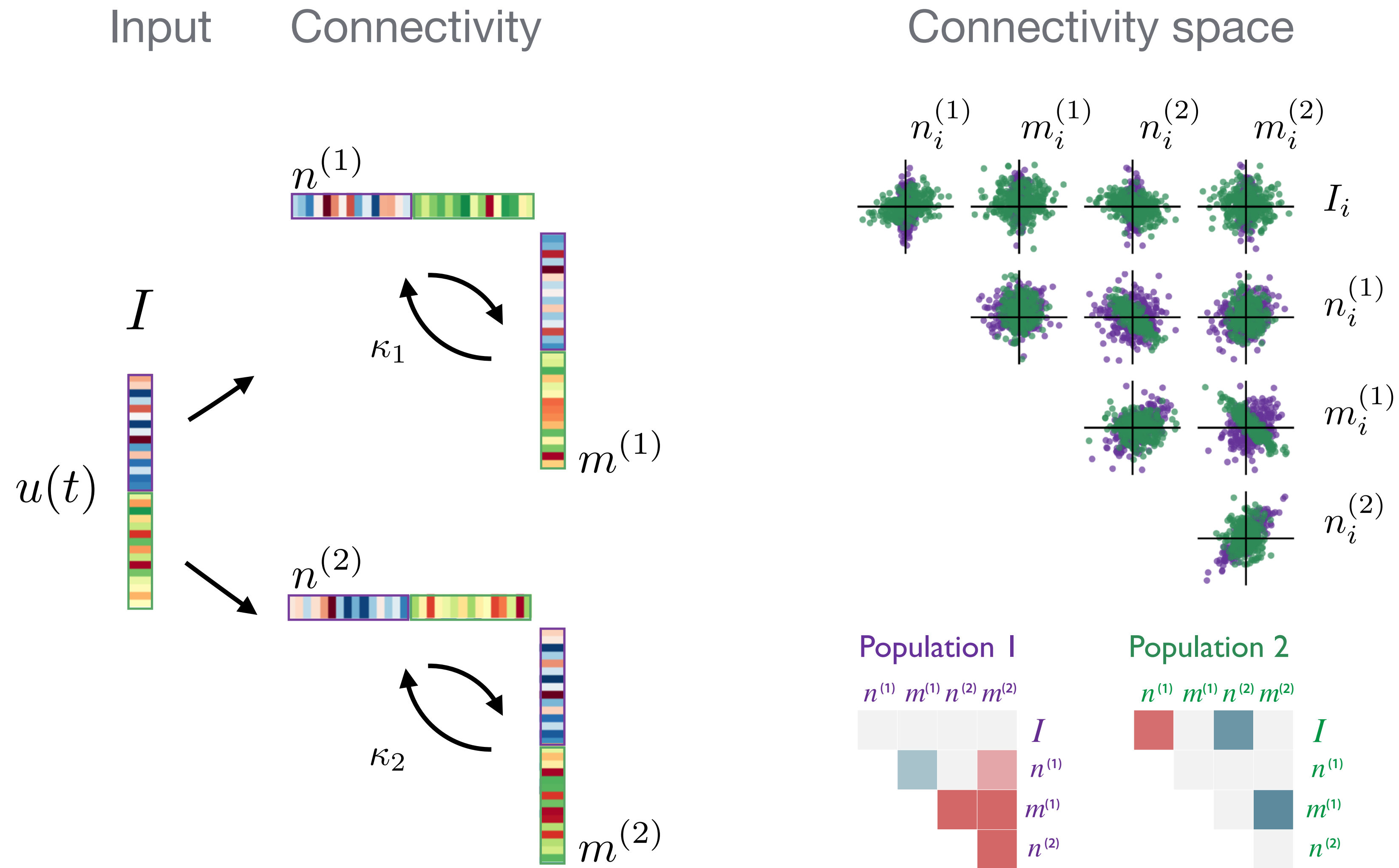


Gaussian distribution: $\tilde{\sigma} = \langle \phi' \rangle \sigma$ $\longrightarrow$ Limited range of dynamical landscapes

Connectivity statistics



Mixture of Gaussians



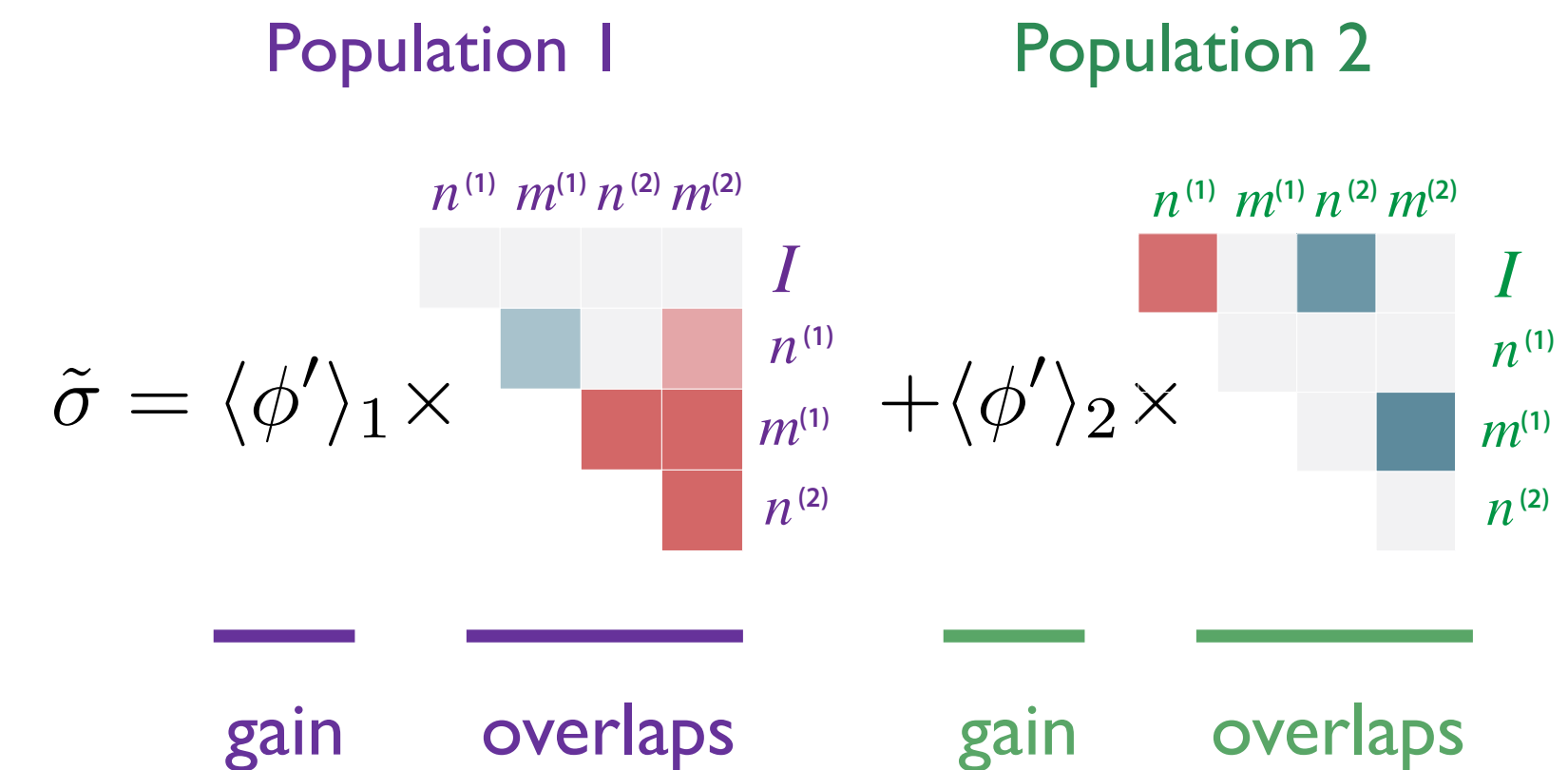
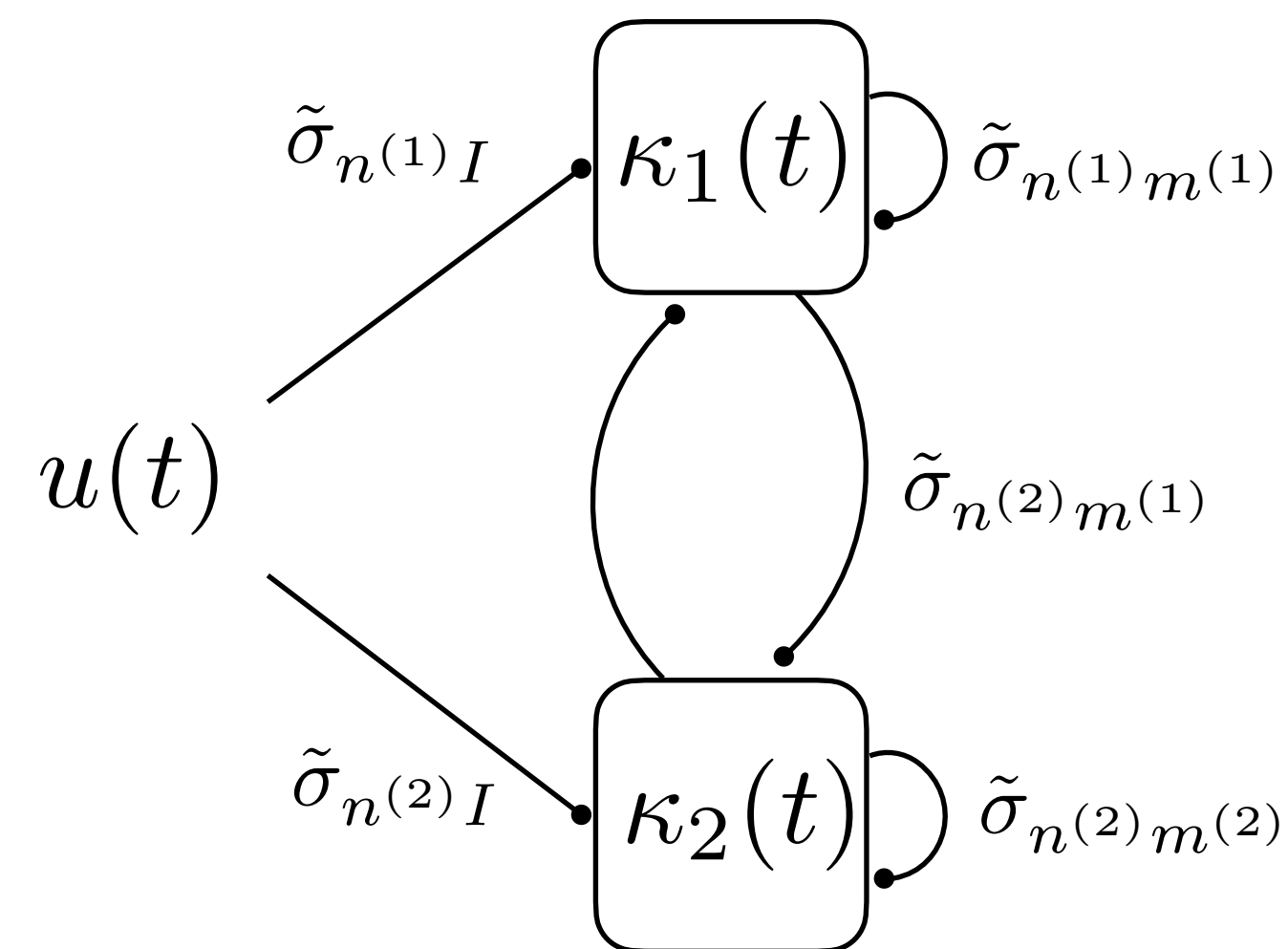
➔ Different interactions for each population

[Beiran et al 2021, Dubreuil*, Valente* et al, 2022]

Effective circuit

Latent dynamics

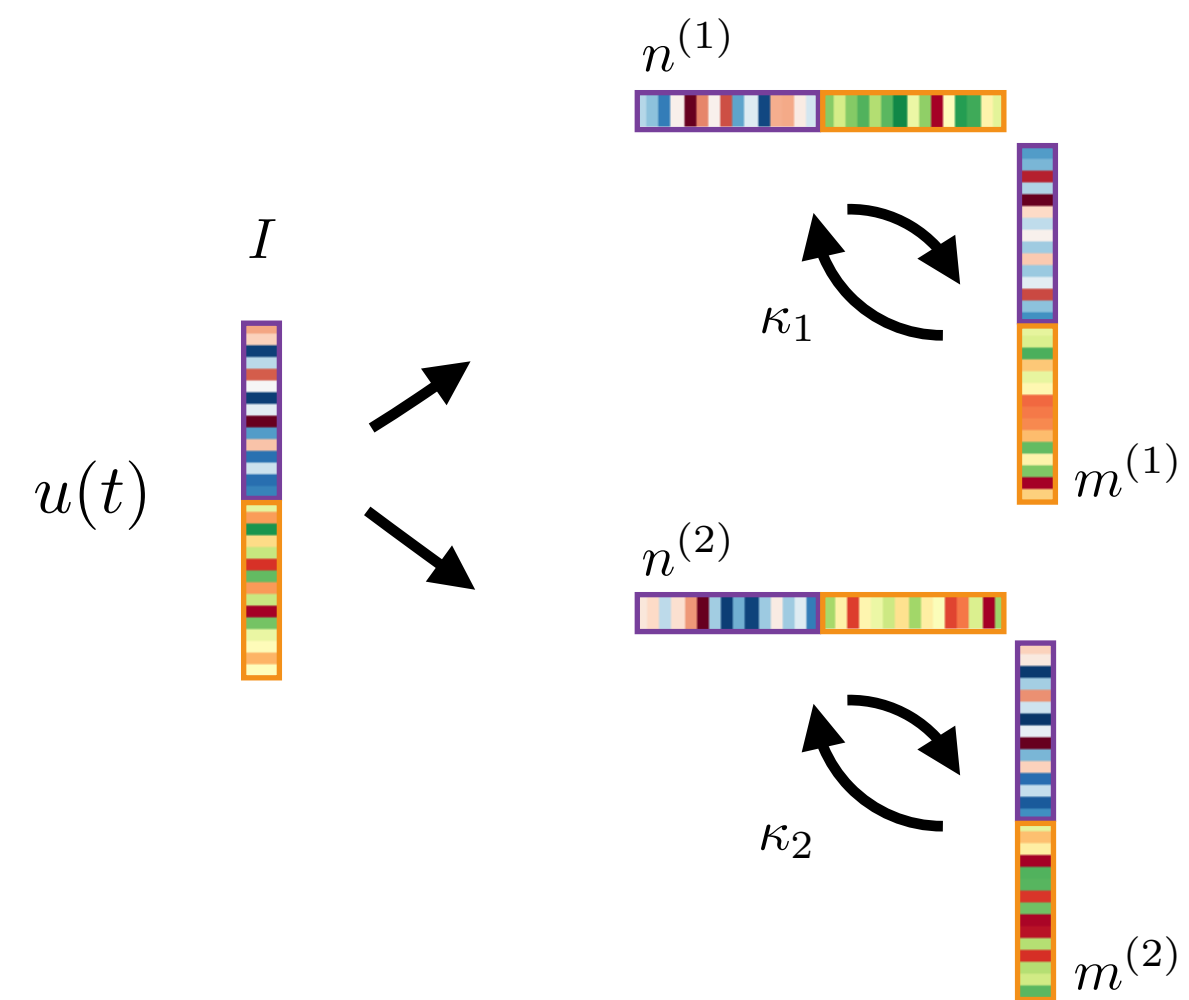
Effective couplings



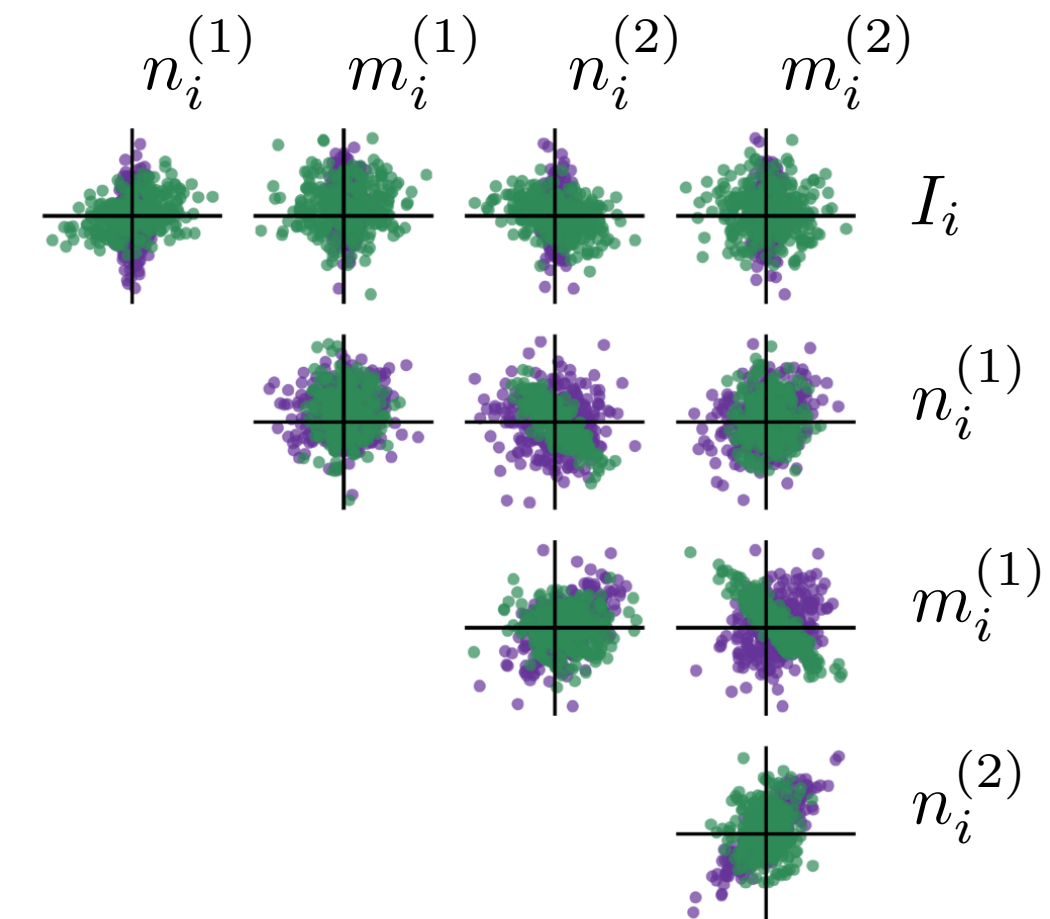
➔ Exact non-linear description + universal approximation theorem [Beiran et al, 2021]

$$\begin{aligned} \frac{d\kappa_1}{dt} &= -\kappa_1 + \tilde{\sigma}_{n^{(1)}m^{(1)}} \kappa_1 + \tilde{\sigma}_{n^{(1)}m^{(2)}} \kappa_2 + \tilde{\sigma}_{n^{(1)}I} u(t) \\ \frac{d\kappa_2}{dt} &= -\kappa_2 + \tilde{\sigma}_{n^{(2)}m^{(1)}} \kappa_1 + \tilde{\sigma}_{n^{(2)}m^{(2)}} \kappa_2 + \tilde{\sigma}_{n^{(2)}I} u(t) \end{aligned}$$

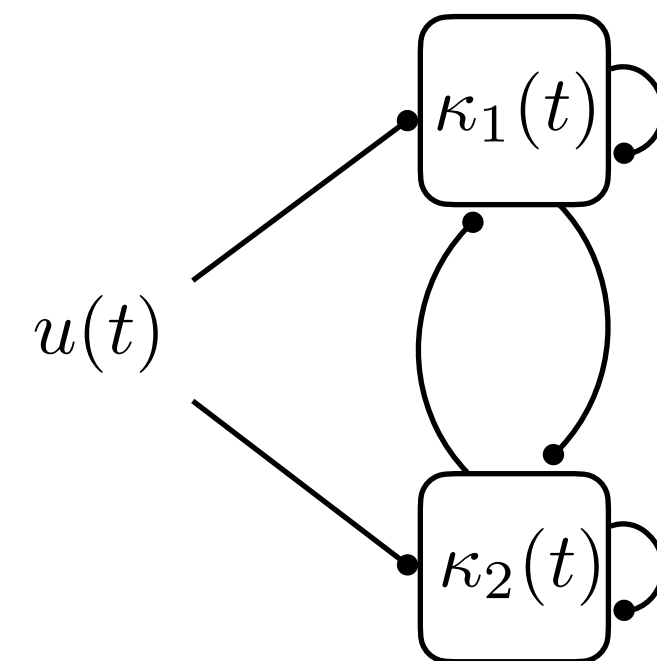
Two types of structure



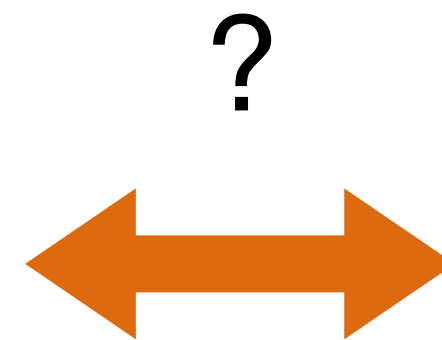
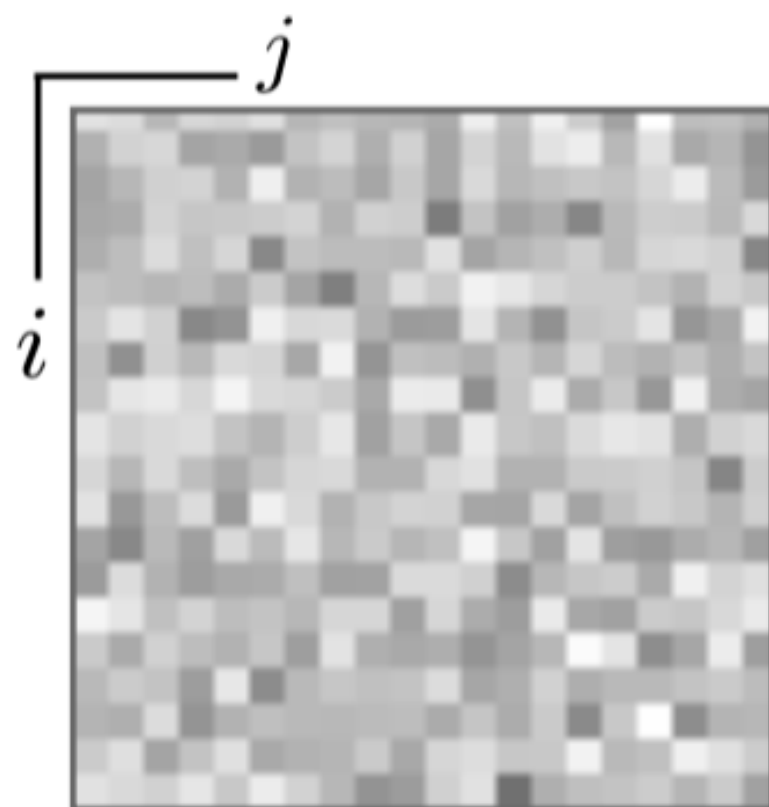
Rank
sets dimensionality



Subpopulations
shape dynamics

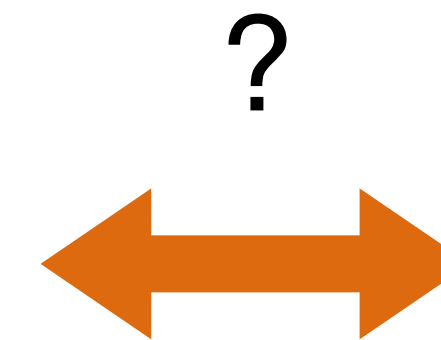
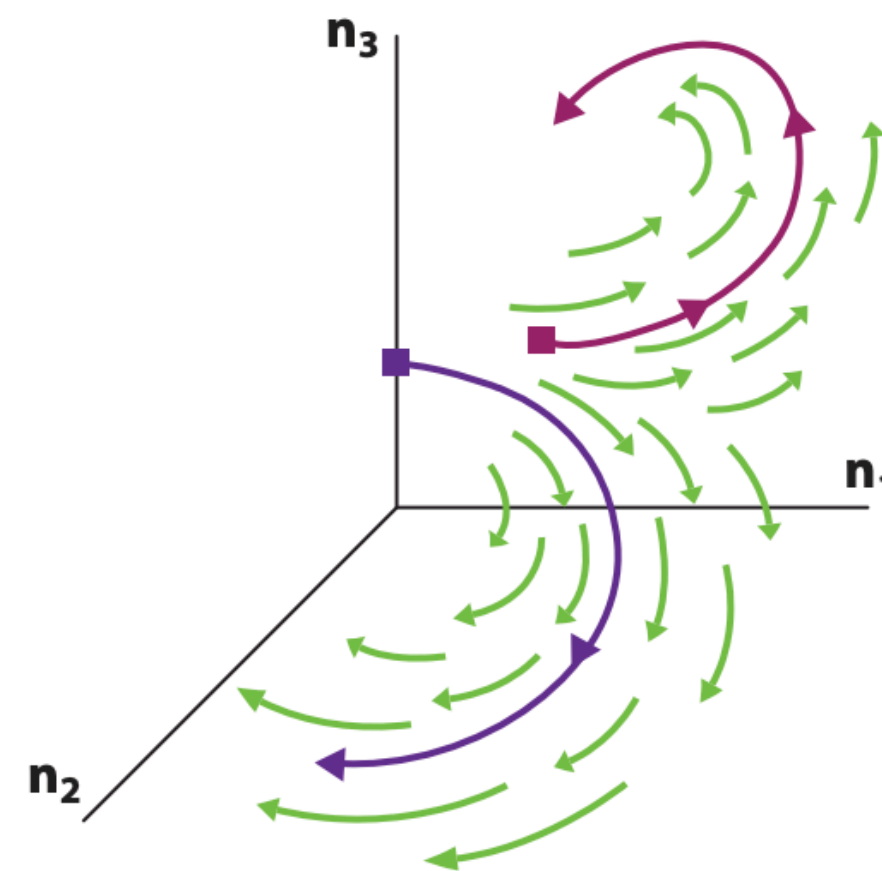


Structure



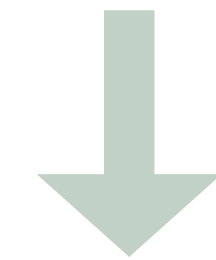
Latent dynamics

$$\frac{d\kappa}{dt} = F(\kappa)$$



Function

Input

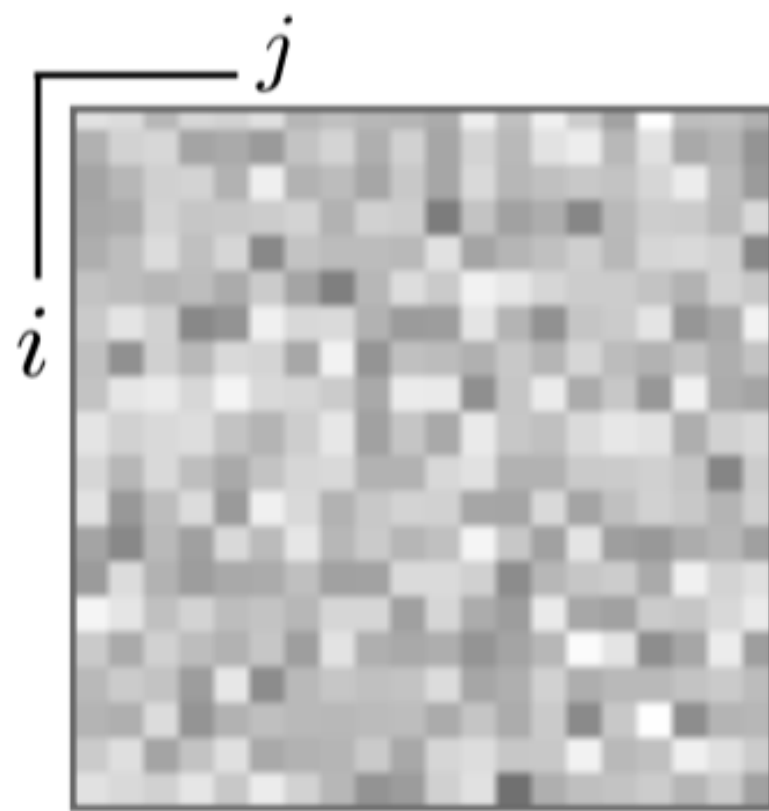


Output

Outline

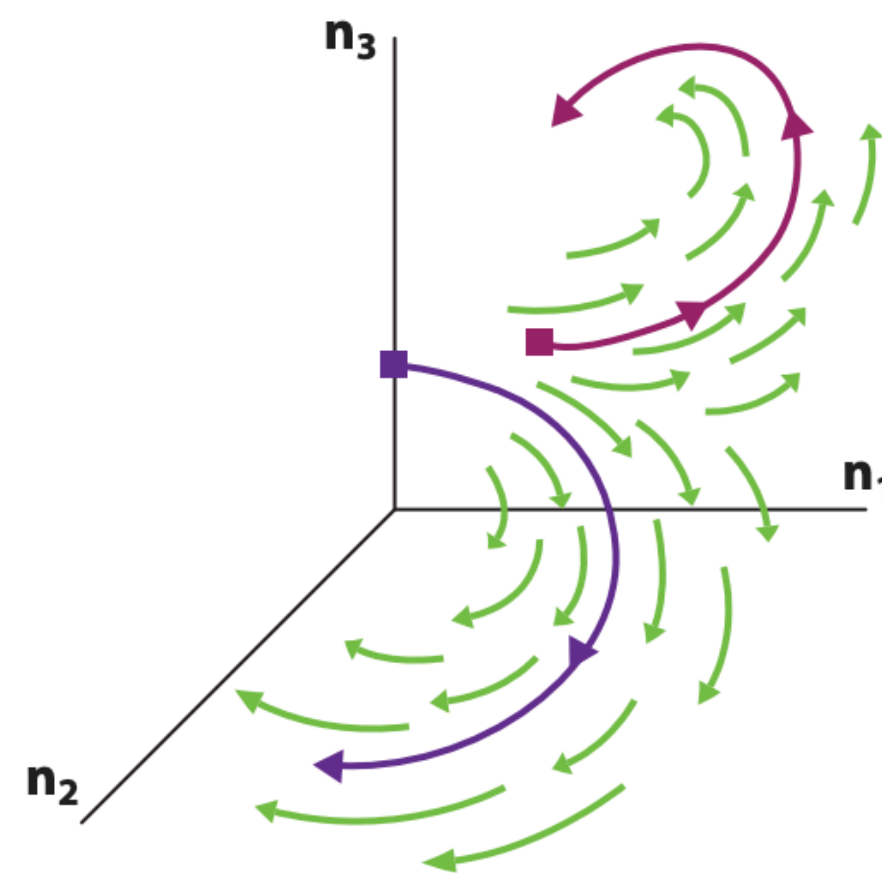
1. Theoretical framework: low-rank recurrent networks
2. Reverse-engineering trained networks

Structure



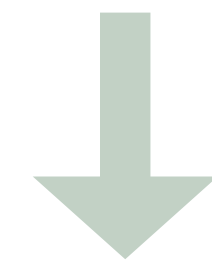
Latent dynamics

$$\frac{d\kappa}{dt} = F(\kappa)$$



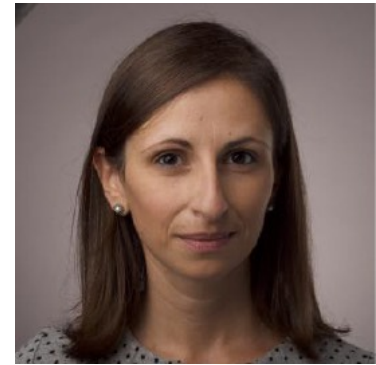
Function

Input



Output

Acknowledgments



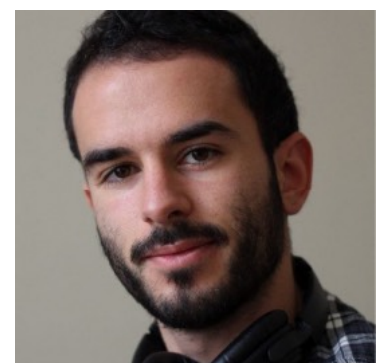
**Francesca
Mastrogiuseppe
(Champalimaud)**



**Manuel Beiran
(Columbia)**



**Alexis Dubreuil
(Bordeaux)**



**Adrian Valente
(ENS)**



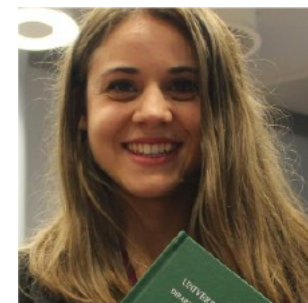
**Friedrich Schuessler
(Technion)**



Joao Barbosa
(ENS)



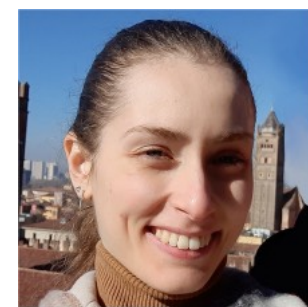
Ljubica Cimesa
(ENS)



Arianna Di Bernardo
(ENS)



Othman Lahrach
(ENS)



Esther Poniatowski
(ENS)



Yuxiu Shao
(ENS)



Giulio Bondanelli
(IST Turin)



Flora Bouchacourt
(Brown)



Elisabeth Herbert
(TU Munich)



Rupesh Kumar
(Maryland)



Josef
Ladenbauer



Daniel Marti
(Amazon)

Collaborators:

Omri Barak (Technion)

Brice Bathellier
(Institut Audition)

Yves Boubenec (ENS)

Jaime de la Rocha (IDIBAPS)

Mehrdad Jazayeri (MIT)

Kishore Kuchibhotla
(Johns' Hopkins)

Jonathan Pillow (Princeton)

Eric Shea-Brown (UW)